An Embedded Ray-Traced Channel Model for ns-3: Architecture, Caching, and Energy-Aware NR Evaluation

Aung Kaung MYAT, Hrishikesh Dutta, Roberto Minerva, Noel Crespi
SAMOVAR, Télécom SudParis
Institut Polytechnique de Paris
Palaiseau, France
Email: {aung-kaung.myat, hrishikesh.dutta, roberto.minerva, noel.crespi}@telecom-sudparis.eu

***Abstract*—For the emerging 6G networks, the testbeds are not ready yet and to deploy such testbed is expensive, time consuming and complex. To overcome these challenges, the simulation approach becomes more important for 6G network research and design. It is very important to have both packet-level protocol realism and site-specific radio propagation model that reflect the real world environment in simulation is very crucial. Ns-3 and 5G-LENA provide mature discrete-event models for traffic, scheduling, mobility, and the NR protocol stack, but their propagation models are analytical or stochastic abstractions that cannot provide site-specific radio propagation information. Sionna RT provides GPU-accelerated ray-tracing for environment-specific channel modelling, but it does not implement end-to-end network protocols. Thus, in this paper, we present *sionnart*, an extension of ns-3 module that embeds Sionna RT ray-tracing engine directly into ns-3 process through pybind11. To make the integration of ray-tracing engine into ns-3 process practical during packet-level simulation, it combine displacement-based coherence cache mechanisms and adaptive virtual receiver generation mechanisms that samples likely future UE positions to reduce number of times it has to perform computationally expensive ray-tracing. To verify that proposed framework is practical, it is evaluated by doing experiments in two parts. First, on IEEE 802.11ax scenarios with up to 32 stations, *sionnart* is $232.3\times$ faster than two server approach of `ns3sionna` in the stationary high-load scenario and it is $43.8\times$ faster under high mobility and high load, while keeping GPU memory below 500 MiB. Second, the proposed framework is applied to simulate 5G NR scenarios with 20 UEs, mixed application traffic, and 2×2, 4×4, and 8×8 gNB arrays across free-space, urban macro, and urban micro environments. Application-related metrics, propagation-related metrics, and energy-related metrics are evaluated for those simulations. The results show that embedded ray tracing can support repeatable system-level studies of the throughput and energy trade-offs created by massive MIMO deployment in geometry-sensitive radio environments.**

***Index Terms*—ns-3, Sionna RT, ray tracing, 5G NR, massive MIMO, energy modeling, propagation cache, network simulation.**

I. Introduction

Sixth-generation (6G) cellular systems are expected to operate in environments where the radio channel is shaped by geometry, blockage, material properties, mobility, and highly directional antenna arrays [5], [14], [15]. Massive MIMO, mmWave and THz access, integrated sensing and communication (ISAC), reconfigurable intelligent surfaces, and AI-assisted beam management all depend on spatial information such as angles, delays, blockage states, and multipath structure [18], [19], [17]. For these use cases, a channel model is not only a scalar path-loss function. It is part of the simulated environment itself.

This creates a methodological gap for network research. System-level simulators are needed because the quantities that users observe – throughput, delay, jitter, packet loss, scheduling decisions, retransmissions, and energy consumption – emerge from MAC, RLC, PDCP, mobility, traffic, and transport behavior. ns-3 provides this discrete-event protocol-stack fidelity through a mature C++ framework and, through 5G-LENA, a detailed 5G NR stack with scheduling, link adaptation, and MIMO support [3], [4]. However, the propagation models commonly used in ns-3, including Friis, log-distance, and standardized stochastic models such as 3GPP TR 38.901, are designed to be computationally efficient abstractions [5], [4]. They are valuable for average-case studies, but they do not reproduce the site-specific propagation paths created by a particular street canyon, indoor layout, or deployment geometry.

Ray tracing addresses this missing physical realism by computing propagation paths directly from a three-dimensional scene [14], [15], [13]. Sionna RT provides a modern GPU-accelerated ray-tracing engine with support for antenna arrays, polarization, and scene-based propagation studies [1], [2]. Yet Sionna RT is not a network simulator: it does not provide the NR protocol stack, packet scheduler, traffic models, mobility procedures, or energy accounting required to study end-to-end network behavior [1], [2], [4]. The central problem is therefore not whether ray tracing or protocol simulation is useful in isolation, but whether deterministic ray-traced channels can be made practical as a live propagation backend for packet-level ns-3 simulations.

Existing integrations typically connect ns-3 to a separate Python ray-tracing process through sockets. The `ns3sionna` implementation follows this design and exchanges channel information with Sionna RT over ZMQ [8]. This separation is flexible, but it introduces per-query inter-process communication, serialization, and duplicated state management. Those costs become especially visible under mobility, where small position changes repeatedly invalidate channel samples while the simulator continues to issue packet-level propagation queries. In this work, we take the opposite design choice: Sionna RT is embedded directly into the ns-3 process, and deterministic channel information is exposed through standard

ns-3 propagation interfaces.

This paper presents `sionnart`, an ns-3 contrib module for fast, repeatable, ray-traced wireless network simulation. The goal is to preserve ns-3 and 5G-LENA as the source of protocol behavior while replacing purely analytical propagation with environment-specific channel information from Sionna RT. The research question is twofold: first, how can the computational overhead of ray tracing be reduced enough for packet-level simulation; second, how does the use of deterministic multipath alter NR performance and energy conclusions compared with conventional ns-3 propagation models?

The main contributions are as follows:

1) **An embedded ns-3–Sionna RT architecture.** We implement `contrib/sionnart`, where the Python interpreter and Sionna RT engine run inside the ns-3 address space via pybind11 [1], [2], [9]. The module connects to ns-3 through `PropagationLossModel`, `PropagationDelayModel`, and spectrum propagation logic, allowing packet-level simulations to request ray-traced path loss, delay, and MIMO channel frequency responses without socket-level round trips.
2) **A coherence-aware propagation cache for mobile simulations.** We introduce a displacement-based cache whose validity radius depends on link distance and transmit-array geometry. The cache stores path loss, delay, LoS state, scalar channel frequency response, and element-level MIMO channel information, while derived port-level and precoded quantities are reused when antenna and beamforming state is unchanged.
3) **Adaptive virtual receiver generation.** To reduce reactive cache misses, the framework predicts short-horizon receiver positions for UEs with stable motion and includes these temporary virtual receivers in the same Sionna RT path-solving call. This converts one ray-tracing invocation into a batch of current and likely future channel samples, improving reuse without adding new network nodes.
4) **A 5G NR performance and energy evaluation methodology.** We extend the simulation with NR gNB and UE energy models driven by PHY state occupancy, antenna-array size, traffic load, and link adaptation [4], [21]. This connects the propagation backend to end-to-end KPIs such as throughput, packet loss, delay, jitter, CQI, MCS, and energy consumption.
5) **Two experiments.** Experiment I compares `sionnart` with pure ns-3 propagation and socket-coupled `ns3sionna` in IEEE 802.11ax scenarios across station counts and mobility/load regimes. At $N = 32$, `sionnart` is up to $232.3\times$ faster than `ns3sionna` in the stationary high-load case and $43.8\times$ faster in the high-mobility high-load case, while keeping GPU memory below 500 MiB. Experiment II applies the embedded backend to 5G-LENA NR scenarios with 20 UEs, 2×2, 4×4, and 8×8 gNB arrays, and free-space, urban macro, and urban micro radio environments [4], [5], [11]. The results show that deterministic multipath changes CQI/MCS selection, application throughput, packet loss, and gNB energy trends, especially in dense urban deployments where reflections, blockage, and array gain strongly affect the link budget.

The remainder of this paper is organized as follows. Section II reviews massive MIMO, stochastic and deterministic channel modeling, ray-tracing simulation, prior integration approaches, ns-3, 5G-LENA, and Sionna RT. Section III describes the embedded architecture, displacement-based cache, adaptive virtual receiver generation, propagation calculations, and NR energy formulation. Section IV evaluates runtime scaling and GPU footprint against pure ns-3 and `ns3sionna`. Section V evaluates the impact of deterministic ray-traced propagation on NR application, propagation, and energy metrics. Section VI concludes the paper and discusses future work toward ISAC-oriented simulation.

II. Background

To understand the motivation behind the integration of ns-3 and Sionna RT, this section reviews the evolution toward Massive MIMO and the need for physically realistic channel models in 6G systems research. The difference between traditional stochastic approaches and deterministic ray-tracing approache is discussed, hightlighting the need for the latter in 6G systems research. Finally, discuss the limitations of prior integration with Simu5G and motivates the use of ns-3 combined with Sionna RT to achieve realistic channel modelling and computationally efficient simulations for advanced 6G research.

A. Massive MIMO for 6G

Multiple-Input Multiple-Output (MIMO) technology is a fundamental building block of 6G systems [11], [17]. Large scale MIMO antenna arrays are essential for operating in high-frequency bands such as mmWave and THz [16], [15]. With the evolution of MIMO antenna arrays, they were able to provide highly directional beamforming and increased angular resolution, allowing the network to distinguish users and objects in environment [12], [19]. This capabilities are important for emerging 6G applications where it has operate in complex, dynamic environment.

In particular, MIMO plays a improtant role in Integrated Sensing and Communication (ISAC), where wireless channel is exploited not only for data transmission but also for environment sensing [18], [19], [20]. In such systems, spatial information of channel such as angles, delays, etc carry information about the physical environment including position and movment of objects. Capturing accurately of these information is critical for enabling functionalities such as beam tracking, object detection, and context-aware decision-making.

However, testing and deploying such kind of systems in real world is challenging due to high cost, complex setup, and limited availability of testing facilities. Simulation become important to overcome these limitation for 6G research. It provide a controlled and scalable environment to evaluate advance MIMO techniques under realistic conditions. However, conventional stochastic channel models, that are widely use in simulators, are designed for statical performance evaluation and lack geometric information required for application such

as ISAC [5], [14], [15]. This limitation movitates the adoption of physically grounded simulation approaches that can accurately represents the interaction between electromagnetic waves and the environment.

B. Stochastic vs Deterministic Channel Modeling

Channel modeling can be broadly classified into two categories, namely stochastic and deterministic channel modeling. Stochastic models rely on satistcial distributions to represent large-scale effects such as path loss and shadowing as well as small-scale fading through multipath clusters [5]. These models are computationally efficient and widely adopted in system-level simulators to evaluate average network performance across standardized deployment scenarios [5], [4].

Eventhough they are suitable for lage-scale evalution, they lack site-specific realistic geometric information about the environment [14], [15]. For instance, a stochastic model cannot distinguish between a line-of-sight path along an open space and blocked path behind a building as both are represented through probalistic parameters. In constrast, deterministic modeling computes the actual electromagnetic interactions within a given 3D environment.

While full-wave solutions based on Maxwell's equations are computationally infeasible, high-frequency approximations such as ray-tracing provide a pratical alternative [14], [15]. Ray-tracing bridges the gap between statistical averages and physical reality by resolving propagation paths including reflections, diffrations and blockages. This is especially important for 6G research as they use mmWave and THz frequency bands, where propagation is highly directional and sensitive to enviorment geometry [16], [15].

C. Ray-Tracing based Simulation

Ray-tracing is a deterministic channel modeling technique that provides a physically grounded representation of radio signal propagation [14], [15]. It models how electromagneticwaves interact with the enviroment by considering refelction, diffrations, and scattering [14], [13].

The fundamental principle of ray-tracing is to approximate electromagnetic waves as rays whose paths are geometrically computed. The channel repose between a transmitter and a receiveris derived by tracing multiple propagation of emitted rays which interact with surronding objects, undergoing reflections, diffractions, or scattering based on the material and geometry of enviroment. Ray-tracing has been widely used sicne 1990s for modeling radio propagation in complex environments [14]. Its deterministic nature enable high accuracy in predicting signal strength, delay spread, and angle of arrival, which are crucial for advanced wireless communication systems [15], [13].

Main advantage of ray-tracing over traditional stochatic models is its ability to provide site-specific realistic channel modeling. This property enables channel evolves smoothly with user modiblity, making it suitable for evaluating advanced scenarios that are difficult to capture with stochatic models. For example, ray-tracing can accurately capture the signal drop when a user moves behind a building, which is difficult to model with traditional stochastic approaches. Furthermore, ray-tracing can provide detailed spatial information about the propagation environment, such as angle of arrival and delay spread, which are crucial for advanced wireless communication systems.

Despite its advantages, ray-tracing is computationally expensive due to the need to evaluate a large number of propagation paths. Recent advances in computing hardware, especially the use of GPUs and parallel processing techniques, have made ray-tracing more practical for large-scale simulations [2], [13], [17].

D. Simu5G and Sionna Prior Integration Limitations

Simu5G is an open-source 5G system-level simulator built on the OMNeT++ framework [6], [7]. It is developed by a consortium of universities and research centers in Spain, led by the Universitat Politècnica de Catalunya (UPC) and has been used for various research works on 5G networks.

Simu5G use Network Description language (NED) for topology and C++ for behavior of the network and applications [6], [7]. Topology is defined in NED file, parameters are passed through a separate omnetpp.ini file and instantiated via a runtime that reflects on those declarations. While elegant for purely C++ network research, the framework is very rigid and hard to integrate with other frameworks such as SionnaRT which is based on Python.

Another key limitation of Simu5G is its lack of native support for multi-input multi-output (MIMO) antenna systems. Recent versions of Simu5G doesn't support MIMO antenna systems natively and require workarounds to enable it which add another layer of complexity and brittleness to the framework.

In addition, it use statistical models for channel modeling which are not as accurate as ray-traced models that reflect the physical environment [7], [14], [15]. Propagation loss and delay are modeled stochastically, which can be acceptable for large-scale system-level analysis but is inadequate for high-accuracy simulations of dense urban or indoor environments. Despite our previous efforts to integrate SionnaRT with Simu5G for MIMO antenna systems with ray-tracing capabilities, we encountered several limitations that motivated our transition to ns-3 for our 6G simulations [7], [1], [2], [3], [4]. Transitioning to ns-3 significantly provide multiple advantages including easier integration with other frameworks such as SionnaRT which is based on Python, native support for MIMO antenna systems and ability to use thirty party libraries like 5G LENA NR for 5G NR simulations.

E. ns3

ns-3 is an open-source, discrete-event network simulator developed in C++ with support for integration with Python [3]. It is an end-to-end network simulator, alternative to OMNeT++, and is widely used in academic for research of communication networks. ns-3 is developed to reflect the reality as close as possible with core concepts and abstractions that map closely to how the real communication network are built.

In ns-3, a `Node` is a fundamental network entity. It serves as a container for `Application`, `Protocols`, and `Network Devices`. Each node can run an `Application` which is a user program that generate packet flows. Ns-3 has several built-in traffic generator for different applications such as video streaming, social media, gaming, VoIP, FTP, etc.

A `Network Device` is an entity that connected to a `Channel` which represent the transmission medium between the devices. The broad community has continuously contributed to the network simulator, which now supports different channel models such as Ethernet, Wi-Fi, LTE, WiMax, etc. [3]. Each time a node sends a packet, the `Channel` calls a statistical or stochastic model, such as Friis, Log-Distance, or 3GPP `PropagationLossModel` and `Propagation DelayModel`, to calculate the path loss and transmission delay for that particular transmission [3], [5].

Moreover, ns-3 provide mobility model such as `ConstantPositionMobilityModel` and `RandomWalk2MobilityModel` to simulate the movement of devices. However, the propagation models provided in ns-3 are limited to statistical models and support for 5G NR and beyond is not supported yet in main framework.

F. 5G LENA NR Module

5G-LENA NR is an extension of ns-3 developed by CTTC for simulating 3GPP compliant 5G NR cellular networks across both sub-6 GHz and mmWave frequency ranges (0.5 to 100 GHz) [4]. The module implements 5G NR core network components, protocol stack including the Physical (PHY), Medium Access Control (MAC), Radio Link Control (RLC), and Packet Data Convergence Protocol (PDCP) layers. The MAC layer in 5G-LENA supports flexible schedulers (e.g., Proportional Fair, Round Robin) for both TDMA and OFDMA-based access, while the PHY layer handles advanced MIMO models and MCS (Modulation and Coding Scheme) adaptation [4], [11], [12].

While 5G-LENA NR provides the protocol stacks for 5G NR, it still rely on the standard ns-3 propagation models which cause the same limitations previously discussed [4], [5]. Specifically, it lacks the site-specific channel information related to environment that it is deployed in. Such information are important to model different research scenarios such as ISAC in 6G network systems. This gap in physical-layer realism is the final motivator for us to integrate a ray-tracing engine into 5G-LENA NR.

G. Sionna RT

Sionna version two is a hardware-accelerated differentiable open-source library for research on communication systems [1], [2]. It is built on top of Mitsuba 3 and Dr.JIt while the API is written in PyTorch unlike the original Sionna library. It is composed of `Sionna RT` which is a stand-alone ray tracer for radio propagation modelling, `Sionna PHY`, a link-level simulator for wireless and optical communication and `Sionna SYS`, system-level simulation functionalities based on physical-layer abstraction [1], [2].

However, it lacks the functionalities to simulate the whole 5G NR protocol stack [1], [2], [4]. This is where ns-3 comes in. Sionna RT provides the realistic channel modelling which reflect the propagation of radio waves in a given environment while ns-3 provides the functionalities to simulate the remaining part of the 5G NR protocol stack. To integrate the Sionna RT into ns-3, we leverage pybind11, allowing Sionna RT to live directly within the ns-3 process [9], [1], [2]. This allows the simulator to calculate spatially consistent CIRs with minimum overhead, effectively replacing stochastic abstractions with deterministic physical calculations.

III. METHODOLOGY

The overall architecture of the proposed integrated simulation framework is depicted in Figure 1. The proposed integrated simulation framework follows a layered design in which ns-3 remains the primary discrete-event simulator while Sionna RT is embedded as environment aware ray-tracing engine [9], [8], [1], [2]. The main goal of the architecture is to preserve the mature protocol, traffic generation, scheduling capabilities of ns-3, Sionna RT handle ray-tracing which provide realistic site-specific propagation channel model, while leveraging GPU acceleration for computation intensive process

At the top is the application layer where the user defined traffic are generated between UEs and gNB. These traffic can be of different type such as video streaming, social media, gaming, FTP and VoIP. This traffic generators allow to simulate different workload scenarios which reflect the realistic network traffic behavior.

The middle layer colored in green is the network layer where the ns-3 stack is located. Node manangement, mobility, routing, scheduling are handled by this layer. It also handle MAC layer operation according to the chosen medium access and protocol standard. For instance, the way packet processed for Wi-Fi 802.11ax or 5G NR is different and it is handled by the ns-3 MAC layer. When a packet is generated by the application layer, it is first passed to the ns-3 stack. Then, the ns-3 stack processes the packet according to the protocol stack. Finally, the packet is transmitted over the wireless medium where it call `PropagationLossModel` and `PropagationDelayModel`, in order to calcualte path loss, transmission delay and channel fade.

When the call to `PropagationLossModel` and `PropagationDelayModel` triggered, the compuation of propagation delay and path loss is handled by the Sionna RT engine which provide environment aware channel information. However, compuation these information using ray-tracing every time packet traverses is computationally intensive and it can significantly slow down the simulation. To address this challenge, the next layer handle this issue using caching mechanism. Whenever, a packet traverses between node A and node B, it check in cache for the channel information using their positions. If the nodes are in positions where the calculated dispalcement distance is below the threshold value, it retrieve the channel information from cache as channel information relatively unchanged if they are under the coherence distance. Otherwise, it trigger the ray-tracing

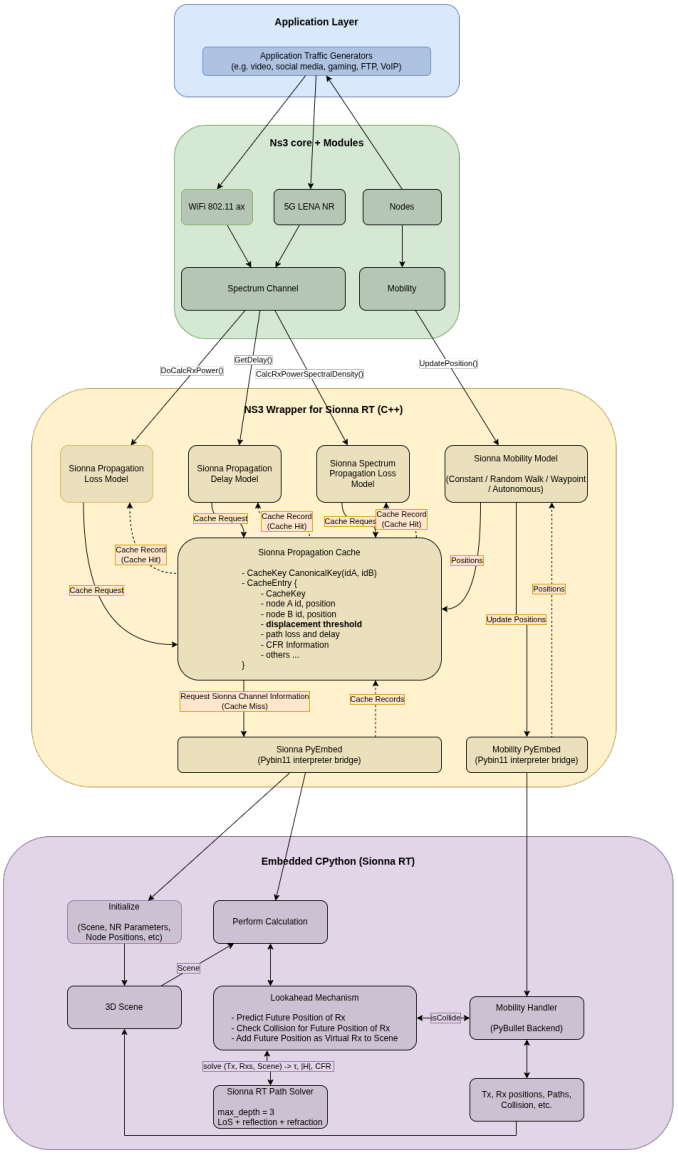


Fig. 1. Architecture of the integrated simulation framework.

engine to compute the channel information and update the cache with the new information. In order to maximize the caching efficiency and reduce number to times it has to call the ray-tracing engine, future possible positions of the nodes are generated based on their current position and speed.

Then these generated positions are added to the scene as virtual receviers in additional to real receviers and the paths calculation is done by Sionna RT engine for all receviers (real and virtual). The channel information obtained for virtual receviers are added to the cahce for future use in addition to the channel information of the real positions. This help to remove the need to call the ray-tracing engine every time packet traverses. However, ns-3 can not directly call the Sionna RT engine to compute the channel information as they are written in different programming languages. So, wrapper class is created to call Sionna RT engine from ns-3 in this layer.

Final bottom most layer which is in purple color is the Sionna RT engine where the ray-tracing is performed to compute the channel information. Before the simulation starts, the radio propagation parameters, antenna configuration, and 3D scene information are initlized by passing the information form ns-3 side. During the simulation, when the channel information requested is not in the cache, the calculation for future positions of the node, channel information for each node in different positions is calculated by using Sionna Path Solver. Path solver calculate path loss, time of arrival, angle of arrival, angle of departure, etc based on the rays propagated in the 3D environment [2], [14], [13]. These calculation are done on GPU for efficient computation which give substantial performance. Finally, the calculated channel information is returned to the ns-3 side through the wrapper class and updated in the cache.

A. Sionna Propagation Cache Mechanism

1) Displacement-Based Cache Validity: Performing the calculation using ray-tracing on Sionna RT is an computationally intensive calculation. The goal of the cache mechanism is to reduce this computationally expensive calculation for every ns-3 propagation query. When the receiver moved only a small distance from the previous position, the propagation information does not change significantly. By using this idea, the cache validation is implemented based on spatial displacement rather than a fixed time interval. In this section, how the cache is implemented is described in detail.

The cache stores the records of the each transmitter and receiver pair for ns-3 simulation. Each cache entry is indexed by a canonical unordered node pair,

$$\kappa(a,b) = (\min(I_a, I_b), \max(I_a, I_b)), \tag{1}$$

where I_a and I_b are ns-3 node identifiers. During the runtime, the cache stores the node positions $\mathbf{x}_a^0$ and $\mathbf{x}_b^0$, path loss, transmission delay, channel frequency reqponse (CFR), and the state for path whether it is in Line of Sight (LoS).

The distance between Tx and Rx is calculated using

$$d_{a,b}^0 = \left\|\mathbf{x}_a^0 - \mathbf{x}_b^0\right\|_2, \tag{2}$$

and the validity radius can be calculated using

$$\Delta_{a,b} = \max\left(\Delta_{\min}, \alpha \frac{0.886 d_{a,b}^0}{N_{\text{tx,col}}}\right). \tag{3}$$

where α is set to 0.4 and $\Delta_{\min}$ is set to 0.1 m as default values.

When the query for propagation delay and path loss happens for the Tx-Rx pair, the current posisitons $\mathbf{x}_a(t)$ and $\mathbf{x}_b(t)$ are compared against the trace-time positions. A valid cache entry is found if

$$\left\|\mathbf{x}_a(t) - \mathbf{x}_a^0\right\|_2 < \Delta_{a,b} \quad \text{and} \quad \left\|\mathbf{x}_b(t) - \mathbf{x}_b^0\right\|_2 < \Delta_{a,b}. \tag{4}$$

. Ifboth inequalities are satisfied, the propagation delay and path loss and other data are returned directly. Otherwise, the cache lookup is considered as a cache miss and the snapshot is refreshed.

2) Snapshot Refresh: When the cache miss happen, it does not referesh only one link between Tx and Rx. Instread, it refreshes the cache for all Tx-Rx pairs. First, positions of every node which are using `SionnaMobilityModel` are synchronized into the Python Sionna scene. Then it performs one batched calculation using ray-tracing for all Tx-Rx pairs by invoking `perform_calculation` in `sionnart.py`. This is designed in such a way to reduce overall computation using ray-tracing by calculating for all Tx-Rx pairs at once. When the calculation are completes, it converts the returned records through pybind, and inserts them as new cache records in ns-3 propagation cache. For each record, the cache stores

$$\left(\tau_{a,b},\, L_{a,b},\, \mathbf{h}_{a,b},\, \mathbf{H}_{a,b},\, \mathbf{f},\, \mathrm{LoS}_{a,b},\, \mathbf{x}_a^0,\, \mathbf{x}_b^0,\, \Delta_{a,b}\right), \quad (5)$$

where $\tau_{a,b}$ is delay, $L_{a,b}$ is path loss, $\mathbf{h}_{a,b}$ is the scalar CFR, $\mathbf{H}_{a,b}$ is the MIMO CFR, and $\mathbf{f}$ is the frequency vector. The old cache records are removed only if the new results contains usable records.

3) Lookup and Fallback Behavior: ns-3 query propagation related information for each Tx and Rx pair. For a normal link, the cache lookup sequence is that it search the canonical key, test displacment validity, and return the first valid entry it found. If no valid entry exists, the cache refreshes the full records and retris the same lookup sequence.

In Sionna RT, the link between nodes with same radio role, such as Tx–Tx or Rx–Rx, are not computable. In such cases, the dummy entry with large path loss and delay are returned instead. If Sionna RT does not return usable data for the requested link after refresh, the cache falls back to stochastic Friis model and constant constant-speed delay are returned for that lookup.

Moreover, before calling Sionna RT to refresh the cache, it checks estimated received power using Friis in order to avoid unnecessary computation expensive calculation. If

$$P_{\mathrm{Friis}}^{\mathrm{rx}} + M < P_{\mathrm{weak}}, \quad (6)$$

where $P_{\mathrm{weak}} = -95$ dBm and $M = 6$ dB by default, the link is treated as too weak to justify ray tracing. In that case, Friis loss and constant-speed delay are used directly.

4) Derived MIMO Caches: The cache records returned from Sionna side stores the normalized element-level MIMO CFR. However, ns-3 needs derived quantities such as port-level channel, matrices, PSD-scaled matrices, and effective gains after precoding. These quantities depend not only on geometry but also on antenna objects, beamforming vectors, PSDs, and precoders. Thus, these information are computed based on the returned element-level MIMO CFR and other information from the ns-3 side, and stored in the propagation entry as derived quantities.

The calculation can be done as followed. For a given beam configuration, the element-level channel is projected onto antenna ports. For resource block q, transmit port p_t, and receive port p_r, the port-level channel has the form

$$G_{p_r,p_t}[q] = \sum_{i\in\mathcal{E}_{p_r}} \sum_{j\in\mathcal{E}_{p_t}} w_{r,i}\, w_{t,j}\, \widetilde{H}_{i,j}[q], \quad (7)$$

where $\mathcal{E}_{p_r}$ and $\mathcal{E}_{p_t}$ are the receive and transmit element sets associated with the ports, and $w_{r,i}$ and $w_{t,j}$ are the beamforming weights. This port CFR is cached using antenna identifiers, beamforming-vector hashes, port counts, and link direction.

For spectrum propagation, the port CFR is scaled by the input PSD:

$$C_{p_r,p_t}[q] = \sqrt{S[q]}\, G_{p_r,p_t}[q], \quad (8)$$

where $S[q]$ is the PSD value on resource block q. The most recent PSD-scaled matrix is cached using a PSD hash and PSD size.

When a precoding matrix $\mathbf{W}[q]$ is provided, the effective gain per resource block is

$$g[q] = \sum_{p_r} \sum_{s} \left| \sum_{p_t} G_{p_r,p_t}[q] W_{p_t,s}[q] \right|^2. \quad (9)$$

These gains are cached using the port-CFR key, precoder hash, precoder dimensions, and output resource-block count. This avoids repeated beamforming and precoding reductions while the underlying propagation entry remains valid.

B. Adaptive Virtual Receiver Generation

The cache mechanism mentioned in the previous section reduce repeated call to Sionna RT but it is reactive. It means a new ray-tracing calculation is requested only after a cache entry becomes invalid. The adaptive virtual receiver generation is a proactive mechanism that is used to reduce the number of cache misses. During cache refresh, a small number of temporary receivers are placed ahead of a UE along its recent direction of travel, so that the same Sionna call returns both the current channel and likely near-future channel samples. These receivers are not new network nodes. They are just spatial samples for the same UE. The primary objectives is to reduce the call to computation expensive Sionna RT ray-tracing call by sampling future possible positions of each UE along their trajectory and perform ray-tracing including those virtual positions in a single call. The overhead to call SionnaRT from ns-3 is longer than computing multiple receivers positions in a single call. Thus, the trade off is made in order to minimize the total simulation time.

Let the UE position and speed be

$$\mathbf{x}_r(t) = [x_r(t), y_r(t), z_r(t)]^{\mathsf{T}}, \qquad v_r. \quad (10)$$

. Prediction of virtual receiver positions is not performed for stationary receiver, i.e., when $v_r \le 10^{-6}$. But for a moving UE, the distance to the nearest transmitter is

$$d_r(t) = \min_{b\in\mathcal{B}} \|\mathbf{x}_r(t) - \mathbf{x}_b\|_2 . \quad (11)$$

The valid displacement Δ_r is computed using the coherence formula from Section III-A1, based on $d_r(t)$ and a default $\Delta_{\min} = 0.5$ m. The current sample is predicted to expire after

$$T_{\mathrm{expire},r} = \frac{\Delta_r}{v_r}. \quad (12)$$

This duration represents the expected time before the channel sample expires, which occurs when the UE travels beyond the valid displacement Δ_r at its current velocity v_r. If $T_{\mathrm{expire},r} < T_{\min}$, with $T_{\min} = 1$, only then virtual receivers are generated.

The generation of virtual receivers is closely tied with the underlying mobility model of the UE. For UE with predictable trajectories, such as those use a Waypoint mobility model, the direction of motion is stable and virtual receivers are reliably generated along the projected path. In contrast, for unpredictable movement patterns, such as a random walk, predicting future positions is unreliable and difficult.

To differentiate these scenarios, the framework evaluates trajectory stability by requiring the dot product of the current and previous directions to satisfy

$$\mathbf{u}_r(t-\Delta t)^{\mathsf{T}}\mathbf{u}_r(t) \geq \rho_{\min}, \tag{13}$$

where $\rho_{\min} = 0.7$ where the prediction direction is defined by the normalized horizontal displacement,

$$\mathbf{u}_r(t) = \frac{[\Delta x_r(t), \Delta y_r(t), 0]^{\mathsf{T}}}{\sqrt{\Delta x_r(t)^2 + \Delta y_r(t)^2}}. \tag{14}$$

. If this stability condition is not met, no virtual receivers are generated. This prevenets from wasting computatinal resources on ray-tracing calculations for locations the UE is unlikely to visit.

For stable trajectories, the spacing and horizon distance are

$$s_r = \beta\Delta_r, \qquad D_r = v_r T_h, \tag{15}$$

with $\beta = 1.1$ and $T_h = 3$ s. The number of virtual receivers is

$$K_r = \min\left(K_{\max}, \left\lceil \frac{D_r}{s_r} \right\rceil\right), \tag{16}$$

where $K_{\max} = 3$. Their positions are calculated as

$$\mathbf{x}_{r,k}^{\text{virt}}(t) = \mathbf{x}_r(t) + k s_r \mathbf{u}_r(t), \qquad k = 1, \ldots, K_r. \tag{17}$$

Thus, only position along the UE trajectory is sampled, and the receiver height remains unchanged.

C. Propagation Calculation

This is the point where the cache and adaptive virtual receiver generation mechanisms meet. When a cache miss occurs, ns-3 synchronize the current mobility state of the UEs and positions to Python scene. Then any eligible virtual receivers are added to the Python scene by invoking the adaptive virtual receiver generation function. Afer all of that finished, Sionna ray-tracing done over all receivers including the virtual receivers over the augmented scene. The results is a single geometrically consistent snapshot containing both present and future possible channel records.

Let $\mathcal{R}$ be the physical receiver set and $\mathcal{V}_r$ the virtual receivers generated for UE r. Ray tracing is performed over

$$\mathcal{R}^+ = \mathcal{R} \cup \bigcup_{r\in\mathcal{R}} \mathcal{V}_r. \tag{18}$$

The Sionna `PathSolver` is called with maximum depth 3, line-of-sight paths, specular reflections, refraction, and synthetic arrays enabled; diffuse reflection is disabled [2], [14], [13].

For each Tx–Rx pair (b, r), the exported delay is the minimum valid path delay in nanoseconds,

$$d_{b,r} = \text{round}\left(10^9 \min_{p:\tau_{b,r,p}\geq 0} \tau_{b,r,p}\right). \tag{19}$$

Let the MIMO channel frequency response be

$$\mathbf{H}_{b,r} \in \mathbb{C}^{N_{\text{rx}}\times N_{\text{tx}}\times N_f}. \tag{20}$$

The average channel power and path loss are

$$P_{b,r} = \frac{1}{N_{\text{rx}} N_{\text{tx}} N_f} \sum_{i,j,q} |H_{b,r}[i,j,q]|^2, \tag{21}$$

$$L_{b,r} = -10\log_{10}(P_{b,r}). \tag{22}$$

If $P_{b,r} = 0$, the implementation exports $L_{b,r} = 200$ dB.

The MIMO channel is normalized as

$$\widetilde{H}_{b,r}[i,j,q] = \frac{H_{b,r}[i,j,q]}{\sqrt{P_{b,r}}}. \tag{23}$$

This separates large-scale attenuation from small-scale frequency and antenna selectivity: $L_{b,r}$ carries the path loss, while $\widetilde{\mathbf{H}}_{b,r}$ carries the normalized MIMO structure. For ns-3 export, the tensor is flattened in the order

$$q \to j \to i, \tag{24}$$

corresponding to resource block, transmit antenna element, and receive antenna element.

After the records are returned to ns-3, the temporary virtual receivers are removed from the Python scene. The ns-3 propagation cache then stores the returned records using the same displacement-validity rule as ordinary receiver samples.

D. Energy Model for 5G NR gNB and UEs

To evaluate the network energy consumption during simulations, the ns-3 energy framework is extended by implementing custom `NrGnbEnergyModel` and `NrUeEnergyModel` modules [21]. These models are implemented by tracking the operational states of the NR spectrum PHY layer which can be repesented as the state space $\mathcal{S} = \{\text{IDLE}, \text{RX\_CTRL}, \text{RX\_DATA}, \text{TX}, \text{CCA\_BUSY}\}$. Then the power draw of the nodes are calculated base on the time it spend on each state.

For the User Equipment (UE), the total energy consumed is computed from its base power profile:

$$E_{\text{UE}} = \int_0^T P_{\text{UE}}(s(t))\, dt = \sum_{s\in\mathcal{S}} P_s^{\text{UE}} \cdot \tau_s, \tag{25}$$

where P_s^{UE} is the power consumption in state s, and τ_s is the cumulative time spent in that state.

By contrast, the Next Generation NodeB (gNB) power model accounts for both a baseline operating load and the scaling effects of its antenna array:

$$E_{\text{gNB}} = \int_0^T P_{\text{gNB}}(s(t))\, dt = \sum_{s\in\mathcal{S}} (P_{\text{fixed}} + N_{\text{ant}} P_s^{\text{gNB}})\tau_s, \tag{26}$$

where P_{fixed} is the constant hardware power draw, N_{ant} is the number of antennas, and P_s^{gNB} is the per-antenna state power.

The time a node spends in active states (τ_s) is driven by the network's link adaptation, which relies on the Signal-to-Interference-Plus-Noise Ratio (SINR). For a receiver i on subcarrier k, the SINR is calculated as:

$$\text{SINR}_{i,k} = \frac{P_{\text{tx}}\, G_{\text{tx}}\, G_{\text{rx}}\, |H_{i,k}|^2}{N_0 W_k + \sum_{j\neq i} P_{\text{tx}}\, G_{\text{tx}}\, G_{\text{rx}}\, |H_{j,k}|^2}, \tag{27}$$

where the core variable is the channel coefficient H, supplied by our propagation model (see Section III-C).

The gNB uses this SINR to select a Modulation and Coding Scheme (MCS). A strong signal allows the use of a more efficient MCS. This higher spectral efficiency (η_i) means data is sent faster, reducing the time the node must spend in the power-intensive transmit state ($\tau_{\mathrm{TX},i}$). Ultimately, this determines the overall energy efficiency, measured as energy per delivered bit:

$$\mathcal{E}_b = \frac{E_{\mathrm{node}}}{\sum_i \eta_i \, W_i \, \tau_{\mathrm{TX},i}}, \tag{28}$$

where E_{node} is the total energy of the UE or gNB.

Crucially, this formulation reveals that the evaluated energy efficiency $\mathcal{E}_b$ is highly sensitive to the calculation of the channel coefficient H. A simple analytical model (like Friis) provides a basic approximation, whereas a ray tracer captures a highly realistic, environment-specific channel. In the second part of the experiment, we evaluates exactly how much this choice of propagation model changes the final energy metrics.

IV. Experiment Part I: Runtime Scaling of the Embedded Ray-Tracing Channel

This section evaluates the computational cost of replacing a stochastic ns-3 propagation model with Sionna RT inside the simulator process. We compare three channel backends: `pure_ns3`, `ns3sionna`, and `sionnart`. The `pure_ns3` backend uses the standard ns-3 stochastic propagation model, while `ns3sionna` [8] communicates with an external Python Sionna RT server via ZMQ. Finally, `sionnart` loads Sionna RT directly into the simulator's address space via pybind11 [1], [2], [9].

To benchmark the computational cost of the three channel backends, the same network toplogy, which include a single IEEE 802.11ax access point and $N \in \{1, 2, 4, 8, 16, 32\}$ stations, and same application traffic are used. The benchmarking is done on IEEE 802.11ax instead of 5G NR as the `ns3sionna` [8] is tested only on IEEE 802.11ax and not on 5G NR. The same network topology and same application traffic profile are used so that total time differences of the simulations are caused by the channel backend rather than by changes in application traffic or node placement. With the same setup, three different scenarios are simulated:

- **stationary_high_load**: static nodes and high packet rate;
- **low_mob_low_load**: slow random-walk mobility and low packet rate;
- **high_mob_high_load**: fast random-walk mobility and high packet rate.

The last scenario is to stress test the ray-tracing because mobility repeatedly invalidates the cache of channels while the high packet rate keeps the simulator issuing propagation queries.

1) Total Simulation Time Scaling: Figure 2 shows the total time taken to finish the simulation over the number of stations N with different channel backends. The y-axis is in logarithmic scale because the total simulation time spans more than four orders of magnitude between the smallest and largest runs. The same results are presented in Table I.

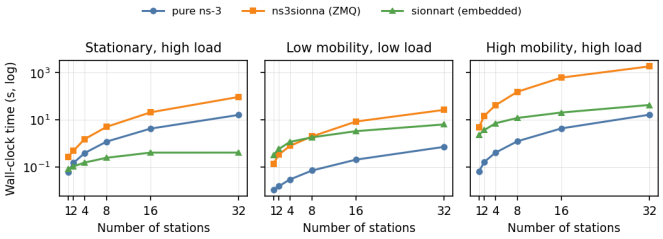


Fig. 2. Total simulation runtime of the three channel backends as the number of stations increases. The communication with external Sionna RT server via ZMQ in `ns3sionna` causes large overhead. The embedded `sionnart` backend avoids the round trip overhead, while the pure ns-3 baseline provides the analytical-channel lower bound.

TABLE I
Total simulation runtime in seconds for representative station counts.

Regime	Backend	$N = 1$	$N = 4$	$N = 16$	$N = 32$
Stationary	pure ns-3	0.061	0.385	4.230	16.160
Stationary	ns3sionna	0.266	1.504	20.826	93.834
Stationary	sionnart	0.082	0.152	0.403	0.404
Low mobility	pure ns-3	0.010	0.029	0.203	0.708
Low mobility	ns3sionna	0.137	0.778	8.453	26.439
Low mobility	sionnart	0.320	1.153	3.292	6.370
High mobility	pure ns-3	0.064	0.400	4.320	16.516
High mobility	ns3sionna	4.821	41.312	622.365	1871.600
High mobility	sionnart	2.293	7.124	20.405	42.772

The scaling behavior of time taken for simulation differs across different scenarios. In the stationary scenario, where stations are not mobile, `sionnart` backend is faster than the `pure_ns-3` baseline once the station count reaches $N = 2$, and at $N = 32$ it is approximately 40 times faster than the baseline. However, this result should not be interpreted as ray-tracing is faster than stochastic Friis propagation model as the reason why it is faster is because of the cache mechanisms. With no mobility, no changes in positions happen so that the ray-tracer is invoked at the initial channel snapshot and the subsequent propagation queries are served by cached channel realizations. In contrast, the pure ns-3 baseline still evaluates the analytical loss model for each packet-level query. Even in this stationary scenario, `ns3sionna` backend is slower than both `pure_ns-3` and `sionnart` backends due to the overhead of the communication between ns-3 and Sionna RT server.

In low-mobility, low-load scenario, the results show the opposite trade-off. When the number of station counts (N) are low, `sionnart` is slower than `ns3sionna` because there are few propagation queries so that the cost of the query outweight the fixed cost of the embedded Python runtime and Sionna scene. As N grows, batching and cache reuse become more effective so that by the $N = 32$ the performance of `sionnart` is 4.15 times faster than `ns3sionna`. It remains 9 times slower than the `pure_ns-3` baseline because the traffic load is low enough that stochastic propagation model remains cheap in term of computation.

The high-mobility, high-load scenario is the most important one because it is the most realistic scenario for the next generation wireless network. In this scenario, `ns3sionna`

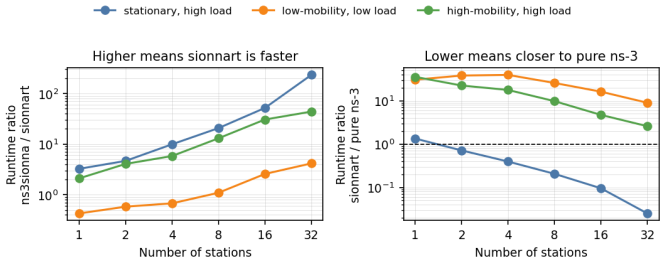


Fig. 3. The left graph shows the ratio of `ns3sionna` runtime to `sionnart` runtime. The right graph is the ratio of `sionnart` runtime to `pure_ns-3` baseline runtime.

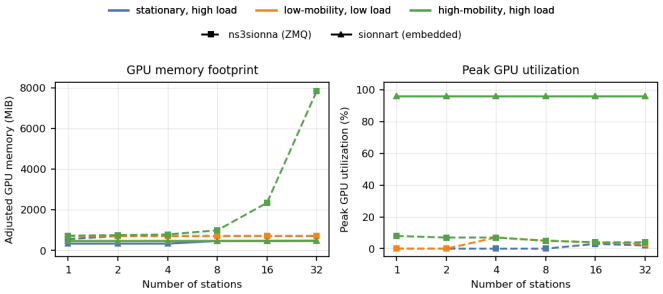


Fig. 4. Reported GPU memory footprint and peak utilization for the two Sionna-backed backends.

TABLE II
RATIOS AT $N = 32$. THE FIRST RATIO IS $T_{\text{ns3sionna}}/T_{\text{sionnart}}$; THE SECOND IS $T_{\text{sionnart}}/T_{\text{pure ns3}}$.

Scenario	Speedup over ns3sionna	Cost relative to pure ns-3
Stationary, high load	232.3×	0.025×
Low mobility, low load	4.15×	9.00×
High mobility, high load	43.8×	2.59×

scales poorly with the number of stations because each cache misses require copying data between processes, waiting for a request and response between two servers, and updating the Sionna side state. The `sionnart` backend still have to pay the price of invoking the ray-tracer for each cache miss, but it avoid the overhead of inter-process communication and keeps the cache inside the same process. As a result, the gap between `sionnart` and `ns3sionna` grows monotonically from 2.1 times at $N = 1$ to 43.8 times at $N = 32$.

2) Speedup Analysis: Figire 3 shows the runtime ratios. The left panel shows $T_{\text{ns3sionna}}/T_{\text{sionnart}}$ and a larger value means that `sionnart` is faster than `ns3sionna`. The right panel shows $T_{\text{sionnart}}/T_{\text{pure ns3}}$ and a value below one means that `sionnart` is faster than `pure_ns-3` baseline. The same results are show in Table II.

According to these ratios, embedding Sionna RT using pybind11 helps to improve the performance of simulation. In two server design, a cache miss must send data from ns-3 to a separate Python process and wait for the reply. In the embedded design, ns-3, the cache, Sionna RT, all run inside the same process. This removes much of the communication cost and the remaining cost is the actual ray-tracing, which is still needed when the experiment uses site-specific propagation.

However, two server design is also have its own merits. Having separate server for Sionna RT allows us to run the ray-tracing engine on a different machine, and it also allows us to run multiple ray-tracing engines in parallel. But for our use case, we only need to run the ray-tracing engine on the same machine as ns-3. So we choose the embedded design.

3) GPU Footprint and Utilization: Figure 4 shows the GPU memory footprint and processing core utilization for each scenario. For `sionnart`, allocated memory is almost flat. It is 337 MiB for the smallest stationary scenario, about 465-469 MiB for most scenarios, and 491 MiB for the largest high-mobility scenario. This is because most of the memory usage is due to the Sionna RT scene and increasing number of stations in the same scene, not the number of packets processed by ns-3. By contrast, `ns3sionna` grows significantly in the high-mobility scenario. The memory increases from 730 MiB at $N = 1$ to 7858 MiB at $N = 32$.

The peak-utilization panel present the maximum observed value but not the average GPU load. `sionnart` utilize upto 96% at its peak for performing the channel calculations but the duration of running at peak is very short. On the other hand, `ns3sionna` has lower peak utilization but the GPU is busy most of the time.

In summary, `sionnart` keeps a nearly fixed GPU memory footprint because the scene and others are reused across channel queries. `ns3sionna` uses much more memory because it has to keep per-link ray-tracing state as the number of mobile links increases. For `sionnart`, it utilize full potential of GPU and duration of running calculation is very short while the `ns3sionna` is busy most of the time with lower peak utilization.

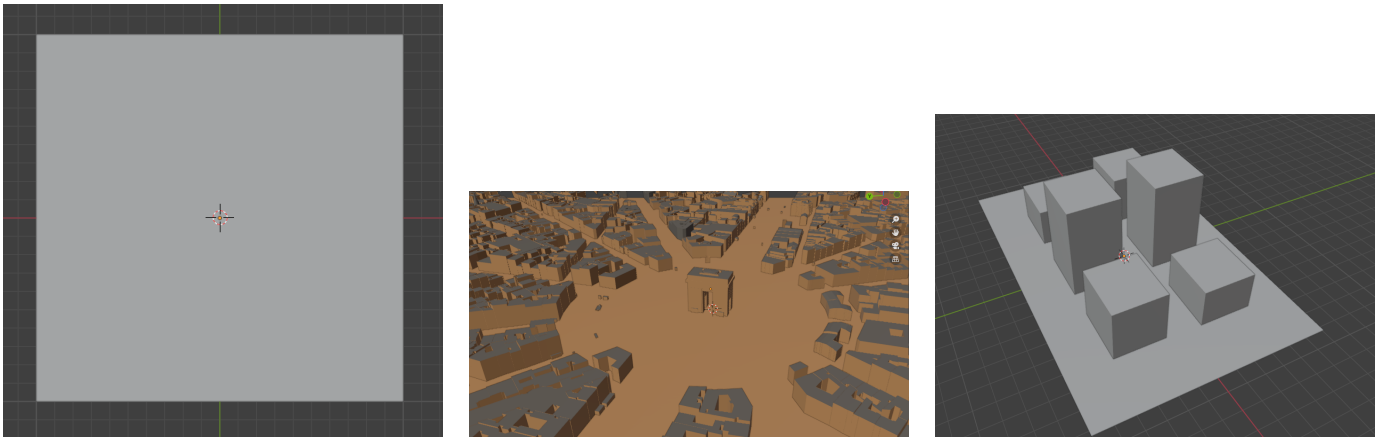
V. EXPERIMENT PART II: NR APPLICATION, PROPAGATION, AND ENERGY EVALUATION

After the embedded approach is validated, we evaluate the performance of the embedded Sionna RT backend in a realistic 3GPP NR deployment [4], [5]. In this section, the performance changes and energy consumption according to different traffic loads and gNB's antenna array sizes are evaluated. The performances are tested on three different radio environments: free space, urban micro, and urban macro.

A. Experiment Configuration

In each simulation run, one gNB and20 UEs are used for simulation. The number of UEs is fixed to 20 due to the computation limitation of GPU and total time required to finish the simulation to be in reasonable range. Different gNB array is varied over 2×2, 4×4, and 8×8 while each UEs uses a 2×2 array. The simulations are tested in three different radio environments:

- **free space**: 2 GHz carrier, 35 m gNB height, and UE distances from 50 m to 1000 m;
- **urban macro**: 3.5 GHz carrier, 25 m gNB height, and UE distances from 35 m to 150 m;
- **urban micro**: 28 GHz carrier, 10 m gNB height, and UE distances from 10 m to 80 m.



(a) Free-space scenario. (b) Urban macro scenario. (c) Urban micro scenario.

Fig. 5. The three simulated radio environments: (a) free space, (b) urban macro, and (c) urban micro.

TABLE III
APPLICATION DATA RATE BY APPLICATION AND LOAD.

Application	Flows	Low		Medium		High	
		DL	UL	DL	UL	DL	UL
Video	12 DL / 12 UL	24.000	0.300	96.000	1.200	300.000	3.750
VoIP	2 DL / 2 UL	0.128	0.032	0.256	0.064	2.000	0.500
Social	2 DL / 2 UL	1.000	0.050	10.000	0.500	40.000	2.000
Gaming	1 DL / 1 UL	0.100	0.025	1.000	0.250	10.000	2.500
FTP	3 DL / 3 UL	3.000	0.150	30.000	1.500	150.000	7.500
Total	20 DL / 20 UL	28.228	0.557	137.256	3.514	502.000	16.250

The height of gNB antenna is set to different values depending on the radio environments. The UE positions are generated randomly in the given environments and the distance between gNB and UEs are set according to the environments. The free-space scenario is the baseline, the urban macro scenario represents a sub-6 GHz deployment, and the urban micro scenario represents the most demanding physical setting as it combines a mmWave carrier with geometry-sensitive short-range links. The 3D visualizations of these three environments are shown in Figure 5.

For scheduling UEs, a TDMA proportional fair scheduler, and full-buffer TDD slots are used. On top of the realistic wireless channel model, different applications with different traffic loads are tested. The traffic profile includes video, voice over IP (VoIP), social media, gaming, and FTP. The number of UE for each application is set to 12(80%), 2(10%), 2(10%), 1(5%), and 3(15%) for DL, and 12(80%), 2(10%), 2(10%), 1(5%), and 3(15%) for UL, respectively as mentioned in [22]. The data rate for each application according to load levels are summarized in Table III.

B. Free-Space Scenario

The free-space scenario is used as the baseline scenario because it don't have any building blockage and makes the distance as the dominant large-scale factor. The UEs are randomly spread over a wide radius up to 1000m from the gNB. As a result, the scenario contains both favourable near links and very weak cell-edge links.

1) Application-related metrics: Figure 6 shows the application-related metrics for the free-space scenario. The downlink flows are dominated by high-rate applications such as video streamin,g social media, gaming and FTP. The stochastic ns-3 channel gives higher downlink throughput than Sionna RT in this long range scenario. At 8×8 antenna array configuration, ns-3 delivers 4.85 Mbit/s with 21.57% packet loss, while Sionna RT delivers 3.77 Mbit/s with 26.62% packet loss. The same trend is visible at 2×2, where ns-3 delivers 4.77 Mbit/s and Sionna RT delivers 3.52 Mbit/s.

Overall, the performance improve slightly as the antenna array size of gNB increases from 2×2 to 8×8. The reason the performance is low in this scenario compared to others is because the cell radius is large relative to the carrier and transmit configurations. Even though the UEs near the gNB were able to achieve higher throughput, the average performance is reduced due to the UEs that are fars.

2) Propagation-related Metrics: The propagation-related results are presented in Figure 7. The average path loss is around 94.2 dB and the estimated receive power is around -48.2 dBm for both ns-3 and Sionna RT modes. The path delay metrics differentiate two models. The ns-3 reports about 3059 ns path delay while Sionna RT reports about 2237 ns. Both are within the expected range for hundreds of metres of radio path length and the difference reflects the different delay calculations used by the two propagation backends.

In both model, CQI and MCS improve with gNB array sizes but ns-3 remains more optimistic. At 8×8, ns-3 achieve mean CQI 15 and mean MCS 27 while Sionna RT achieve mean CQI 14.79 and mean MCS 26.59. The gap between ns-3 and Sionna RT is small because most of flows are direct path as there is no obstcales like building in the free-sapce scenario.

Larger array size helps to improve the performance in terms of CQI and MCS because of the beamforming gain and spatial selectivity but they do not change the fundamental fact that some UEs are very far from the gNB.

3) Energy Consumption related Metrics: In Figure 8, the energy consumptions of gNB and UEs by array size, and traffic load are presented. The gNB power consumption increases with array size in both ns-3 and Sionna RT. In ns-3, the gNB consumes 31.16 W for 2×2, 64.58 W for 4×4, and 199.39 W for 8×8. In Sionna RT, the gNB consumes 30.08 W for 2×2, 59.08 W for 4×4, and 173.50 W for 8×8. The power consumption of UE is similar for all configurations because UE array is fixed at 2×2.

The results shows throughput-per-power trade-off is poor especially for larger array sizes in this scenario. The larger array 8×8 consumes roughly six times the gNB power of 2×2 array but the downlink throughput improve only slightly. This is because array gain helps the radio link, but it cannot remove the scheduling and queueing pressure caused by serving many long-distance UEs at high traffic load.

C. Urban Macro Scenario

The urban macro scenarios is based on the Charles de Gaulle Etoile in Paris, which is a dense urban environment with tall buildings and many obstacles. The gNB is placed at the top of the Arc de Triomphe at a height of 25 m and UEs are distributed in a circle of radius 500 m around the gNB. The antennas are configured to use the 3.5 GHz carrier frequency, which is in the sub 6-Hz range. This scenario is one of the challenging scenario for 6G network deployments due to the high density of buildings and obstacles.

1) Application-related Metrics: Figure 9 illustrates the application-related metrics for the urban macro scenario, comparing ns-3 (solid bars) and Sionna RT (hatched bars) across various antenna configurations. Among the traffic types, data-intensive applications such as FTP and video streaming achieve the highest throughput in both simulators. For FTP, ns-3 outperforms Sionna RT at the 2×2 configuration where ns-3 achieve roughly 16.8 Mbps whereas Sionna RT achieve 15.2 Mbps. But as the gNB array size increase to 4×4 and 8×8, Sionna RT perform better than ns-3 and achieve nearly 19.5 Mbps whereas ns-3 achieve 18.5 Mbps. The statistical ns-3 approach performs well generally but Sionna RT approach performs better at larger array size as it can better leverage massive MIMO spatial multiplexing by capturing multipath components such as reflections and diffractions from the buildings and environment. In contrast, for video streaming application, ns-3 consistently achieve higher than Sionna RT across all antenna configurations.

For Sionna RT, higher packet loss for video traffic is observed in downlink which values are 9.5%, 7.6% and 4.6% respectively for 2×2, 4×4 and 8×8 antenna configurations. Ns-3 performs better than Sionna RT for video traffic and for FTP traffic under 2×2 configuration, however, Sionna RT outperforms ns-3 for FTP traffic under 4×4 and 8×8 configurations. Overall, increasing the antenna array size significantly reduce packet loss for most of the applications in both simulators. In the uplink, the packet loss for all most all of the applications are very high for both simulators, ranging from 15% for VoIP to nearly 100% for gaming and social applications in Sionna RT. This reflects the impact of harsh environment conditions on the constrained transmission power of UEs which struggles to overcome severe blockages, resulting in poor uplink condition. Interestingly, Sionna RT still achieve higher throughput than ns-3 for gaming and FTP traffic in uplink eventhough Sionna RT observe higher packet loss than ns-3 for both applications.

Packet delay and jitter metrics further align with these observations. Downlink delay is worst for video and FTP traffic where Sionna RT achieve 295 ms, 220ms, and 175 ms for FTP and 220 ms, 145 ms, and 65 ms for video, respectively for $2 \times 2, 4 \times 4, 8 \times 8$ antenna configurations. Ns-3 also observes high delay for video and FTP traffic in downlink as well, however, it is lower than Sionna RT. The reason Sionna RT experience higher delay is because the detailed multipath components captured by ray-tracing introduce longer propagation paths and varying signal arrival times, increasing the queuing delay. Jitter remains relatively low in both simulators for all the applications but ns-3 occasionally reports high spike for application like FTP and gaming.

2) Propagation-related Metrics: The propagation-related results are presented in Figure 10. Path loss and received power remain consistent across all antenna size in both simulators at roughly 83.5 dB and -37.5 dBm respectively. Both ns-3 and Sionna RT achieve nearly identical values for propagation related metrics but differences can only be observed in path delay, CQI and MCS. Ns-3 estimate a path delay of approximately 220 ns while Sionna RT estimate higher delay around 360 ns across all antenna configurations. Higher delay in Sionna RT was due to taking into consideration for multipath components.

Similarly, lower CQI and MCS values are observed in Sionna RT than ns-3 for all antenna configurations. For instance, in the 2×2 configuration, Sionna RT reports an average CQI of 9.5 and MCS of 16.5 while ns-3 estimates a CQI of 10.5 and MCS of 17.5. Nevertheless, increasing gNB array size from 2×2 to 8×8 improves CQI and MCS in both simulators. For example, Sionna RT reports an average CQI of 10 and MCS of 17 at 8×8 configuration while ns-3 estimates a CQI of 11 and MCS of 18. Improving CQI and MCS according to gNB array size validate that increasing the array size enhances beamforming capabilities and effectively reduces impact of environment and improves overall link quality.

3) Energy Consumption-related Metrics:

In Figure 11, the energy consumptions of gNB and UEs by array size, and traffic load are presented. The power consumption of gNB scales accordingly with both antenna array size and traffic load and both simulators show similar scaling trends. For instance, both ns-3 and Sionna RT reports approximately 25 W, 40 W and 100 W for $2 \times 2, 4 \times 4, 8 \times 8$ array configurations under low traffic load. However, Sionna RT reports slightly higher power consumption than ns-3 for all antenna configurations. The reason is link to lower CQI and MCS values estimated by Sionna RT, which alters resource

block allocations and transmission durations compared to ns-3. These results show that increasing gNB array size improve performance but at the cost of significantly higher energy consumption.

The power consumption of UE remains relatively stable for all antenna configurations in both simulators but scale noticeably with traffic load. The UE power consumption increases from approximately 0.058 W at low load to nearly 0.095 W at high load. Both simulator observe nearly identical UE power consumption across all configurations but Sionna RT consumes slightly higher energy than ns-3. This is because UE are constrained to 2×2 antenna configuration in this experiment.

D. Urban Micro Scenario

The urban micro scenario is designed to simulate the dense street-level environment where gNB is deployed at lower elevation of 10 m, typically below the surrounding rooftops. In this scenario, the likelihood of non-line-of-sight (NLOS) conditions and strong multipath effects due to scattering from nearby buildings and street furniture is higher compared to the macro scenario. The complex propagation environment presents unique challenges for establishing reliable links, especially for high-throughput applications.

1) Application-related Metrics: Figure 12 illustrates the application-level flow metrics for the urban scenario. A comparative analysis between the statistical ns-3 model (solid bars) and the ray-tracing Sionna model (hatched bars) reveals substantial differences in how each simulator resolves the complex 3D urban environment. In terms of downlink throughput, Sionna RT consistently achieve higher throughput for data-intensive applications such as FTP and gaming compared to ns-3. For example, in Sionna RT, FTP throughput reach nearly 20 Mbps with 4×4 and 8×8 gNB antenna array sizes while ns-3 reach around 15 Mbps. This mean that Sionna RT approach capture multipath components (reflections and diffractions) from urban structures which lead to higher downlink throughput than ns-3 approach when MIMO spatial multiplexing is leveraged. Moreover, larger gNB array sizes improve performance is clearly validated because scaling from 2×2 to 8×8 antenna arrays enable to achieve higher througput in both ns-3 and Sionna RT.

The packet loss metrics further emphasize the environmental impact. In downlink, ns-3 get high packet loss for FTP and gaming, nearly 30% in 2×2 configuration, while Sionna RT get lower loss rate for these applications. In the constract, Sionna RT approach have higher loss for video traffic at 2×2 configuration compared to ns-3. In either case, increasing the antenna array size to 8×8 significantly reduces the downlink packet loss to near-zerio across most applications, demonstrating how massive MIMO configurations effectively overcome urban blockages and fading. However, the packet loss remains exceptionally high ranging from 60% to over 90% for both simulators in uplink. This refelcts a fundamental real world limitation that constrained transmission power of UE and smaller antenna array szie struggles to overcome the urban obstacles.

Packet delay and jitter metrics also align with the previous observations. Downlink delay is worst under 2×2 array configuration, with ns-3 experiencing extreme spikes for FTP > 350 ms and Sionna RT showing elevated delays for video and social traffic. The multipath components captured by ray-tracing can introduce variable signal arrival times, occasionally increasing delay and jitter for continuous streams in Sionna RT. However, as the gNB array scales to 8×8, both delay and jitter drop significantly which confirms that massive MIMO is essential for meeting the strict latency and reliability requirements of 6G applications in dense urban areas.

2) Propagation-related Metrics: The propagation-related results are presented in Figure 13. Average path loss, received power remain consistent across all antenna size at roughly 95.5 dB and -62.5 dBm for both simulators. Both ns-3 and Sionna RT report nearly identical values for propagation related metrics.

However, the differences between ns-3 and Sionna RT are clearly visible in the path delay and channel quality metrics. While ns-3 estimate a path delay of approximately 120 ns, while Sionna RT estimate higher delay around 178 ns across all antenna configurations. This difference can be attributed to the fact that Sionna RT approach take into account for multipath components from the 3D urban structures.

Similarly, Sionna RT predicts lower average CQI and MCS values compared to ns-3. For instance, in the 2×2 configuration, Sionna RT reports an average CQI of 6 and MCS of 10 while ns-3 estimates a CQI of 9.5 and MCS of 16.5. The lower CQI and MCS values in Sionna RT highlight the impact of the detailed multipath fading and interference captured by the ray tracer, leading to a harsher but more realistic channel quality assessment. Nevertheless, as the antenna array size scales from 2×2 to 8×8, both CQI and MCS improve significantly in both simulators. This further validates that increasing the gNB array size enhances beamforming capabilities and effectively reduces the harsh urban channel conditions and improves overall link quality.

3) Energy Consumption-related Metrics: In Figure 14, the energy consumptions of gNB and UEs by array size, and traffic load are presented. For the gNB, the power consumption scales accordingly with both antenna array size and traffic load. For low traffic load, both ns-3 and Sionna RT report 23 W, 33 W, and 68 W for $2 \times 2, 4 \times 4, 8 \times 8$ antenna configurations, respectively. However, under high traffic load, the power consumption of the 8×8 array configuration spikes drastically to 215 W in ns-3 and 193 W in Sionna RT. While both simulators show similar scaling trends, Sionna RT reports slightly lower power consumption under high load. The reason is link to the lower CQI and MCS values estimated by Sionna RT, which alters resource block allocations and transmission durations compared to ns-3. These results show that larger array sizes improve performance but at the cost of significantly higher energy consumption.

For the UE, the average power consumption remains relatively stable across different gNB antenna sizes but scales noticeably with traffic load. The UE power consumption increases from approximately 0.05 W at low load to nearly 0.09 W at high load. Sionna RT and ns-3 predict nearly identical UE

power consumption across all configurations. This confirms that the uplink transmission behavior and energy modeling are consistent between both simulators, and that the UEs are operating near their transmit power limits in the challenging urban environment regardless of the gNB's antenna array size.

VI. Conclusion

In order to design and test 6G networks, it is difficult to meet the demanding requirements, there are not many test site that are available and costly to deploy for testing. Thus, simulations become important to design and test 6G systems especially simulation tools that is close to real world channel modelling. In this paper, we presented `sionnart`: an embedded integration of the Sionna RT ray-tracing engine directly into the ns-3 network simulator. The main objective is to replace traditional stochastic channel models with deterministic, physically realistic ray-tracing based framework which captures the complex multipath environments which is crucial for next-generation 6G networks.

However, ray-tracing process is computationally expensive and it is difficult to integrate with network simulator. In order to overcome the computational bottleneck and integration difficulty, we have proposed several novel solutions including an embedded architecture utilizing a displacement-based, coherence-aware cache and a proactive adaptive virtual receiver generation mechanism. Thses desgin choices dramatically reduce the computation overhead and maintain.

The proposed framework is evaluated using two part of experiments. In the first part of the experiment, runtime scaling, integration overhead, GPU memory footprint are evaluated. The proposed `sionnart` achieved up to a $232\times$ speedup compared to the `ns3sionna`, which has two server approach design. It also maintained a flat GPU memory footprint under 500 MiB and even outperformed the analytical Friis baseline in stationary scenarios due to efficient caching mechanism.

In the second part of the experiment, the proposed framework is applied to evaluate 5G NR system in different radio environments such as free space, urban macro, and urban micro. After the simulations, the application-related KPIs such as throughput, packet loss, packet delay, jitter, the propagation-related KPIs such as path loss, path delay, average MCS and CQI, and power consumption related KPIs such as power consumption of gNB and UEs are measured. The results show that ray-tracing based model capture the realistic multipath fading and propagation characteristics better than traditional stochastic analytical models which is crucial for next-generation 6G networks. The results also validated that scaling gNB antenna arrays to larger array size like 8×8 is essential for overcoming the harsh environments condition impact on link quality.

As for future work, we plan to extend the proposed framework to evaluate Integrated Sensing and Communications (ISAC) systems, which is a key enabling technology for 6G networks. Ray-tracing based channel modeling is an essential component for ISAC evaluation because it captures accurate spatial, temporal, and geometric data of the environment, thereby providing more realistic sensing performance assessments. Specifically, the accurate multipath information derived from our embedded ray-tracing approach can be leveraged to jointly simulate communication links and sensing operations such as object detection, tracking, localization and beamforming with tracked objects. This expansion will allow researchers to test and investigate ISAC systems in more realistic environment.

References

[1] J. Hoydis *et al.*, "Sionna: An Open-Source Library for Next-Generation Physical Layer Research," NVIDIA Research. [Online]. Available: https://nvlabs.github.io/sionna/

[2] F. Ait Aoudia, J. Hoydis, M. Nimier-David *et al.*, "Sionna RT: Technical Report," *arXiv preprint arXiv:2504.21719*, 2025. doi: https://doi.org/10.48550/arXiv.2504.21719

[3] The ns-3 Consortium, "ns-3 Network Simulator." [Online]. Available: https://www.nsnam.org/

[4] N. Patriciello, S. Lagen, B. Bojovic, and L. Giupponi, "An E2E Simulator for 5G NR Networks," *Simulation Modelling Practice and Theory*, vol. 96, p. 101933, 2019. doi: https://doi.org/10.1016/j.simpat.2019.101933

[5] 3GPP, "Study on Channel Model for Frequencies from 0.5 to 100 GHz," 3GPP TR 38.901, v17.0.0, 2022.

[6] A. Varga and R. Hornig, "An Overview of the OMNeT++ Simulation Environment," in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008.

[7] G. Nardini, D. Sabella, G. Stea, P. Thakkar, and A. Virdis, "Simu5G–An OMNeT++ Library for End-to-End Performance Evaluation of 5G Networks," *IEEE Access*, vol. 8, pp. 181176–181191, 2020. doi: https://doi.org/10.1109/ACCESS.2020.3028550

[8] A. Zubow, Y. Pilz, S. Rösler, and F. Dressler, "Ns3 Meets Sionna: Using Realistic Channels in Network Simulation," *arXiv preprint arXiv:2412.20524*, 2024. doi: https://doi.org/10.48550/arXiv.2412.20524

[9] W. Jakob, J. Rhinelander, and D. Moldovan, "pybind11 – Seamless Operability between C++11 and Python." [Online]. Available: https://github.com/pybind/pybind11

[10] P. Mogensen *et al.*, "LTE Capacity Compared to the Shannon Bound," in *Proceedings of the IEEE Vehicular Technology Conference (VTC)*, 2007.

[11] B. Bojovic, Z. Ali, S. Lagen, and L. Giupponi, "ns-3 and 5G-LENA Extensions to Support Dual-Polarized MIMO," in *Proceedings of the Workshop on ns-3*, 2022, pp. 1–9. doi: https://doi.org/10.1145/3532577.3532595

[12] B. Bojovic, S. Lagen, and L. Giupponi, "Realistic Beamforming Design Using SRS-Based Channel Estimate for ns-3 5G-LENA Module," in *Proceedings of the Workshop on ns-3*, 2021, pp. 81–87. doi: https://doi.org/10.1145/3460797.3460809

[13] S. Bastos, A. P. Oliveira, and D. T. Suzuki, "Generation of 5G/6G Wireless Channels Using Raymobtime with Sionna's Ray-Tracing," in *Proceedings of the Brazilian Telecommunications and Signal Processing Symposium*, 2023. doi: https://doi.org/10.14209/sbrt.2023.1570923823

[14] A. M. Al-Samman, M. Cheffena, O. Elijah, Y. A. Al-Gumaei, S. K. Abdul Rahim, and T. A. Rahman, "Survey of Millimeter-Wave Propagation Measurements and Models in Indoor Environments," *Electronics*, vol. 10, no. 14, p. 1653, 2021. doi: https://doi.org/10.3390/electronics10141653

[15] C. Han, Y. Wang, and Y. Li, "Terahertz Wireless Channels: A Holistic Survey on Measurement, Modeling, and Analysis," *arXiv preprint arXiv:2111.04522*, 2021. doi: https://doi.org/10.48550/arXiv.2111.04522

[16] A. N. Uwaechia and N. M. Mahyuddin, "A Comprehensive Survey on Millimeter Wave Communications for Fifth-Generation Wireless Networks: Feasibility and Challenges," *IEEE Access*, vol. 8, pp. 62367–62414, 2020. doi: https://doi.org/10.1109/ACCESS.2020.2984204

[17] L. Cazzella, D. Tagliaferri, and M. Mizmizi, "Deep Learning of Transferable MIMO Channel Modes for 6G V2X Communications," *IEEE Transactions on Antennas and Propagation*, vol. 70, no. 6, pp. 4127–4139, 2022. doi: https://doi.org/10.1109/TAP.2022.3169950

[18] W. Zhou, R. Zhang, G. Chen, and W. Wu, "Integrated Sensing and Communication Waveform Design: A Survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1930–1949, 2022. doi: https://doi.org/10.1109/OJCOMS.2022.3215683

[19] Z. Gao, Z. Wan, D. Zheng, M. Matthaiou, and B. Ai, "Integrated Sensing and Communication With mmWave Massive MIMO: A Compressed Sampling Perspective," *IEEE Transactions on Wireless Communications*, vol. 22, no. 3, pp. 1745–1762, 2023. doi: https://doi.org/10.1109/TWC.2022.3206614

[20] H. Hua, J. Xu, and T. X. Han, "Optimal Transmit Beamforming for Integrated Sensing and Communication," *arXiv preprint arXiv:2104.11871*, 2021. doi: https://doi.org/10.48550/arXiv.2104.11871

[21] A. Samorzewski, M. Deruyck, and A. Kliks, "Energy Consumption in RES-Aware 5G Networks," in *Proceedings of IEEE Global Communications Conference (GLOBECOM)*, 2023, pp. 1024–1029. doi: https://doi.org/10.1109/GLOBECOM54140.2023.10437451

[22] Ericsson, "Ericsson Mobility Report," Nov. 2025. [Online]. Available: https://www.ericsson.com/4aca6f/assets/local/reports-papers/mobility-report/documents/2025/ericsson-mobility-report-november-2025.pdf

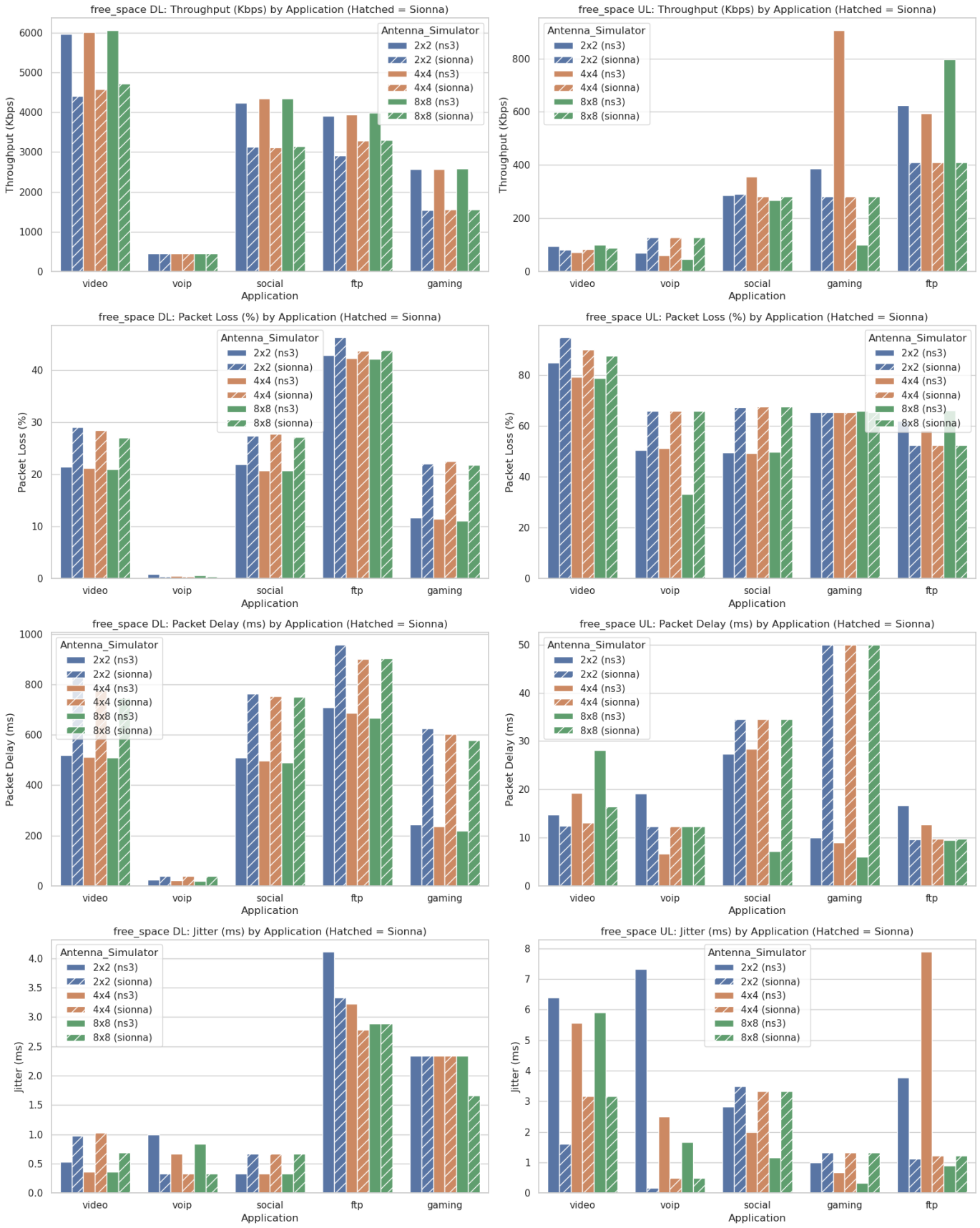


Fig. 6. Free-space Application-related metrics by traffic type, direction, and gNB array size.

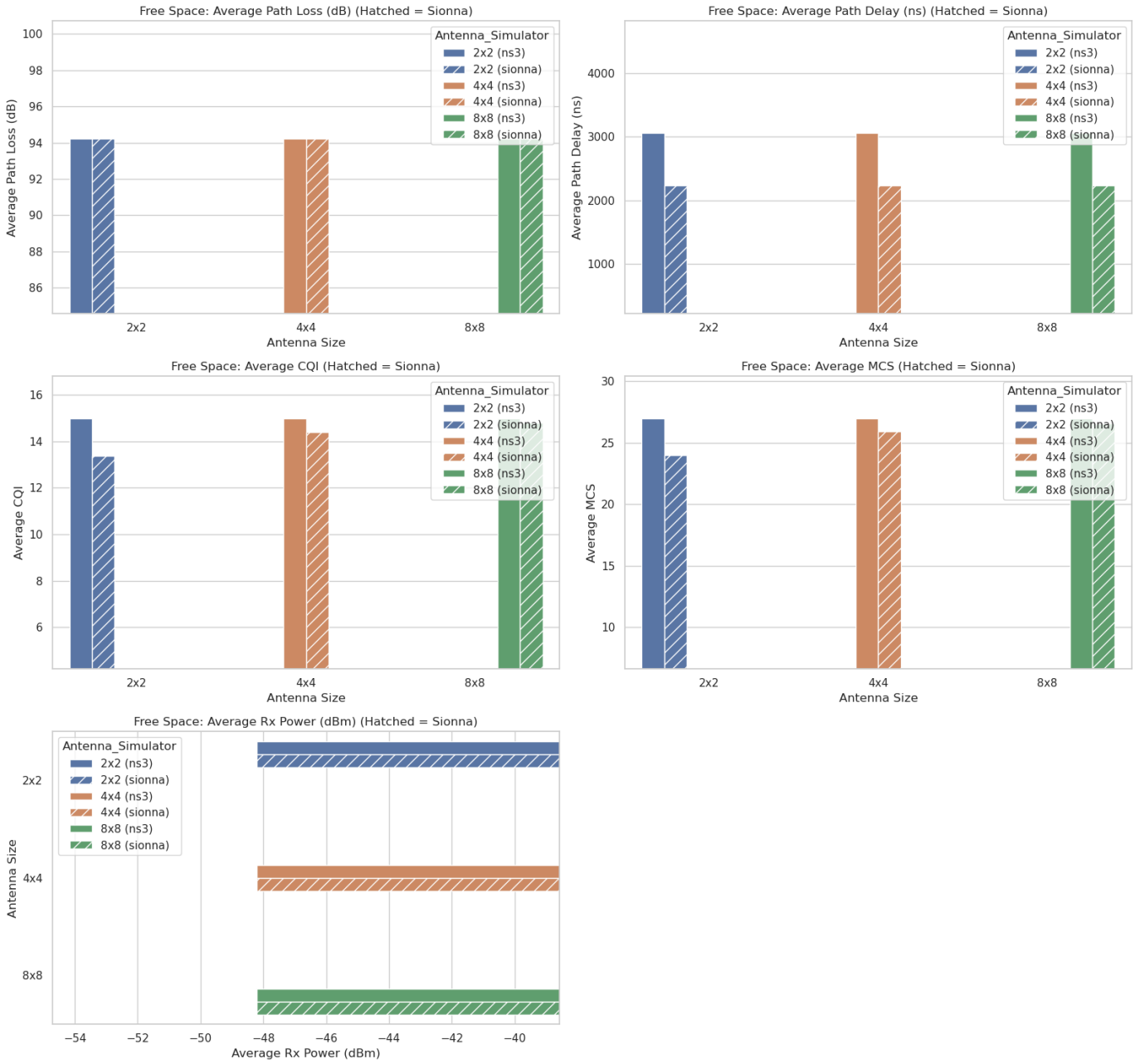


Fig. 7. Free-space propagation-related metrics by channel mode and gNB array size.

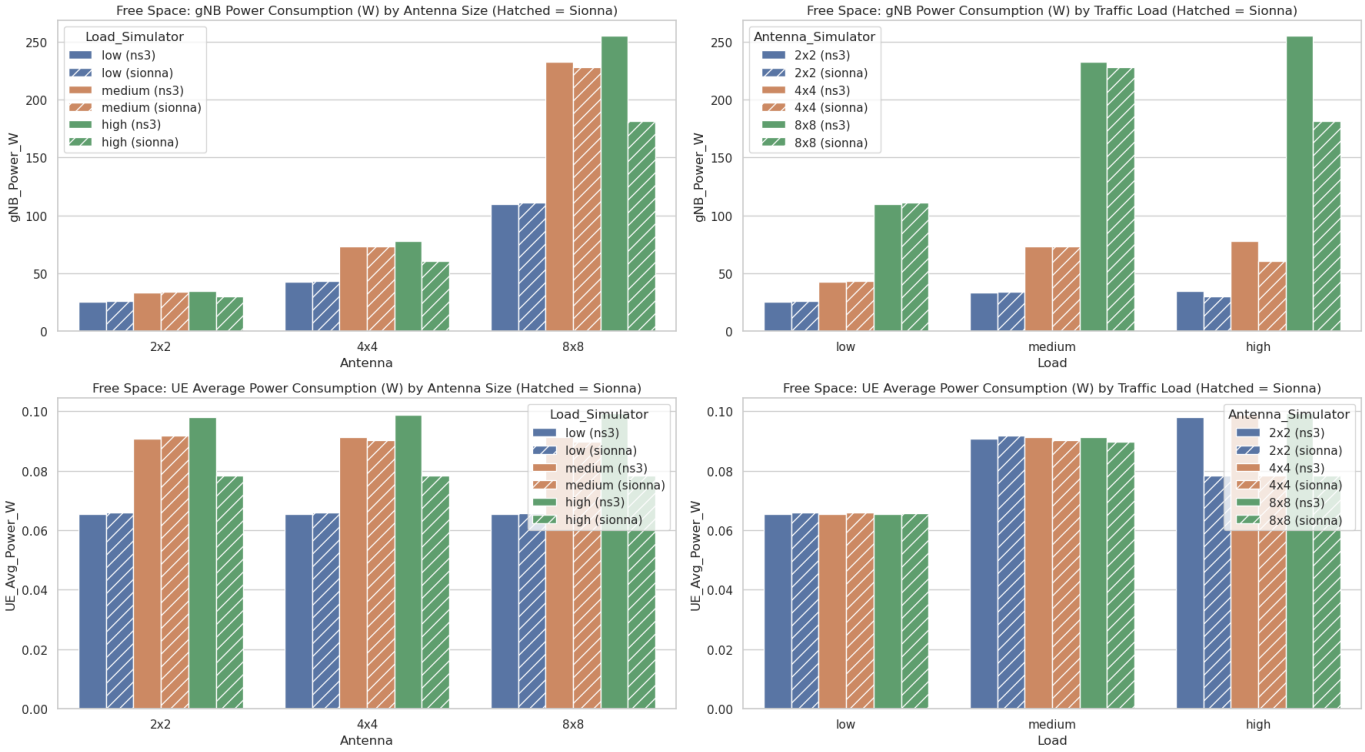


Fig. 8. Free-space energy consumptions of gNB and UEs by array size, and traffic load.

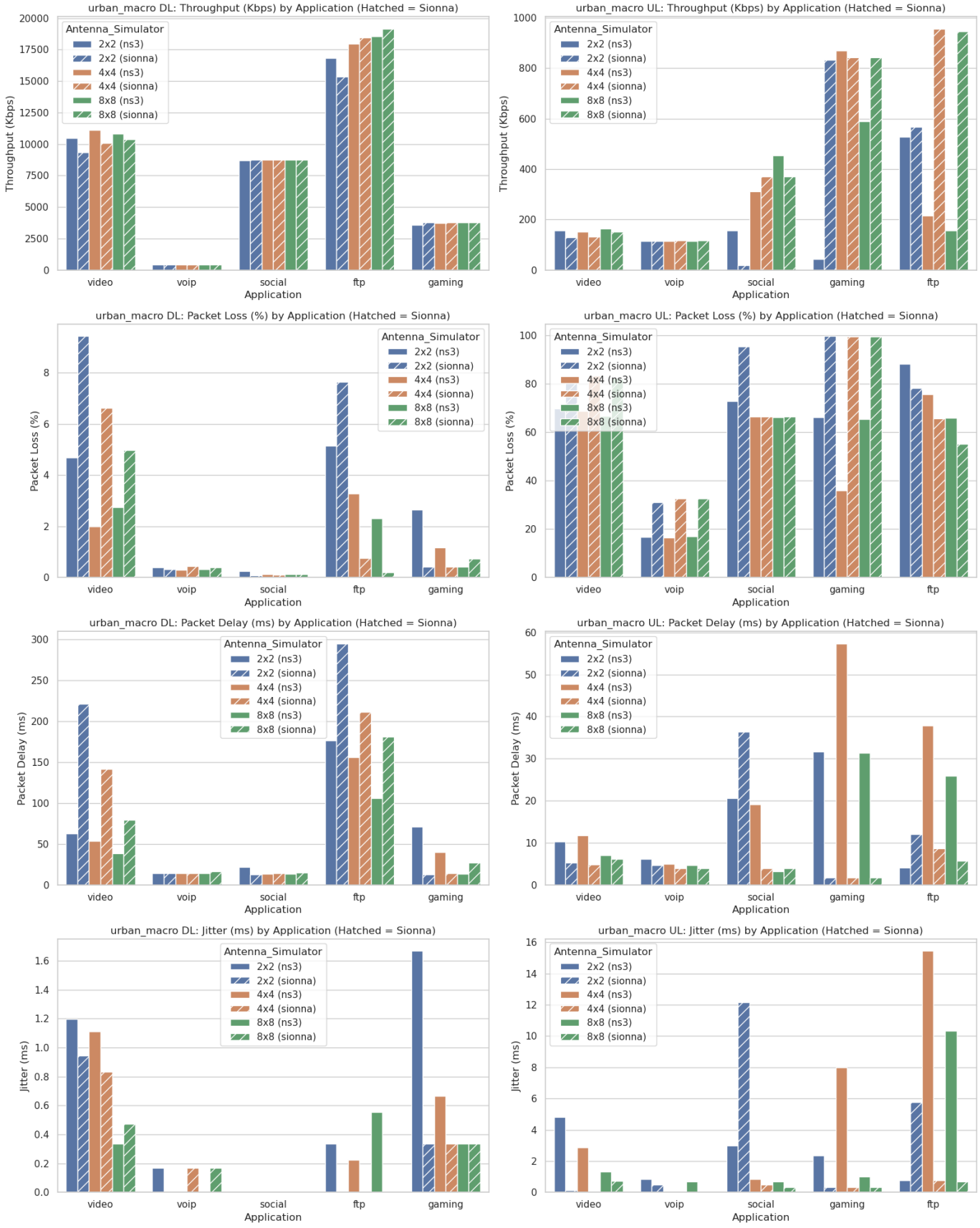


Fig. 9. Urban macro Application-related metrics by traffic type, direction, and gNB array size.

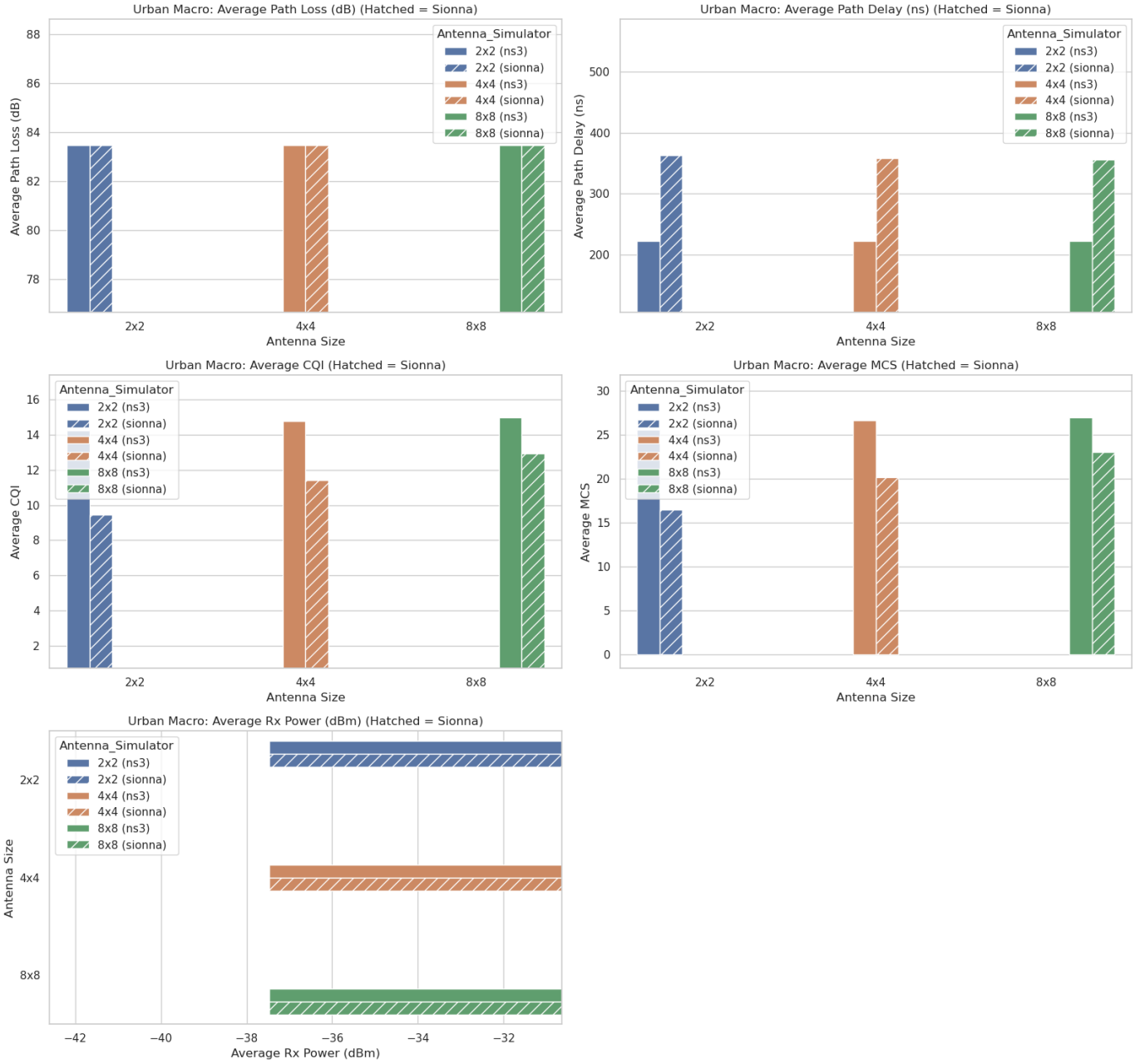


Fig. 10. Urban macro propagation-related metrics by channel mode and gNB array size.

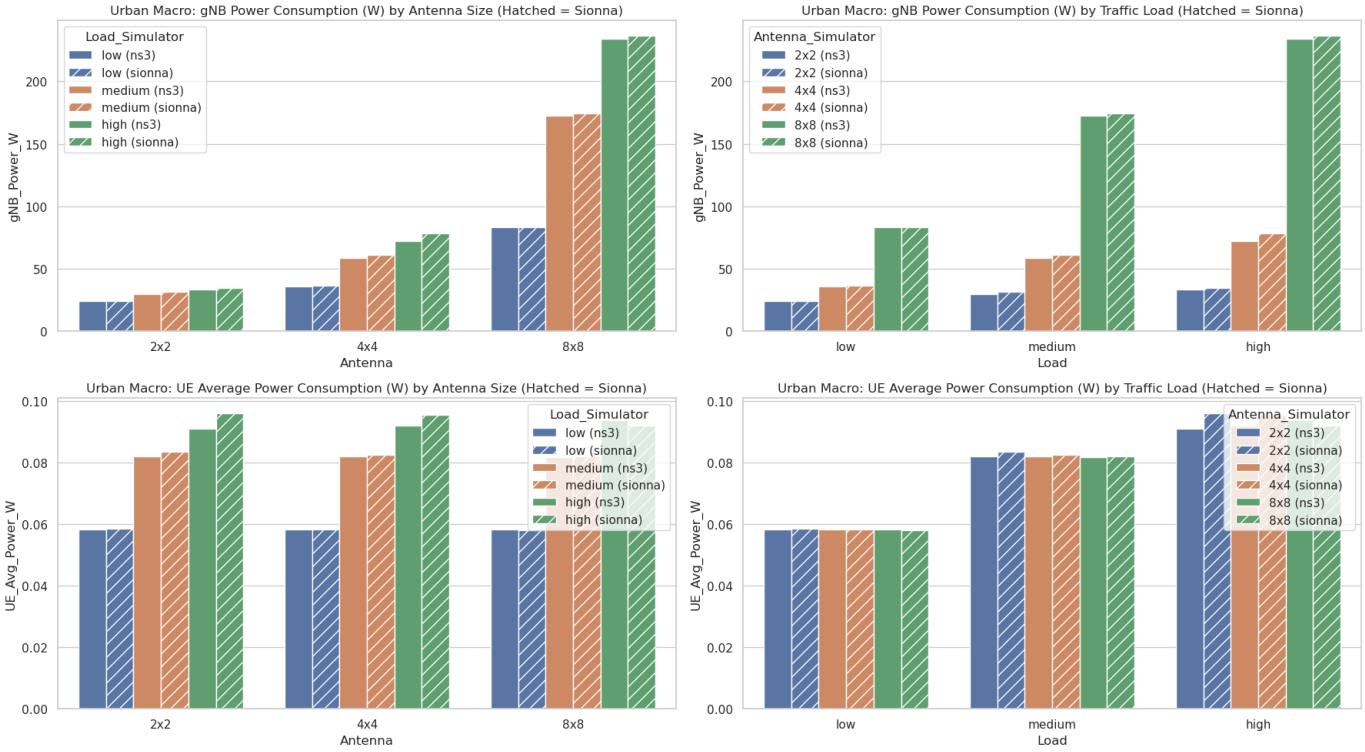


Fig. 11. Urban macro energy consumptions of gNB and UEs by array size, and traffic load.

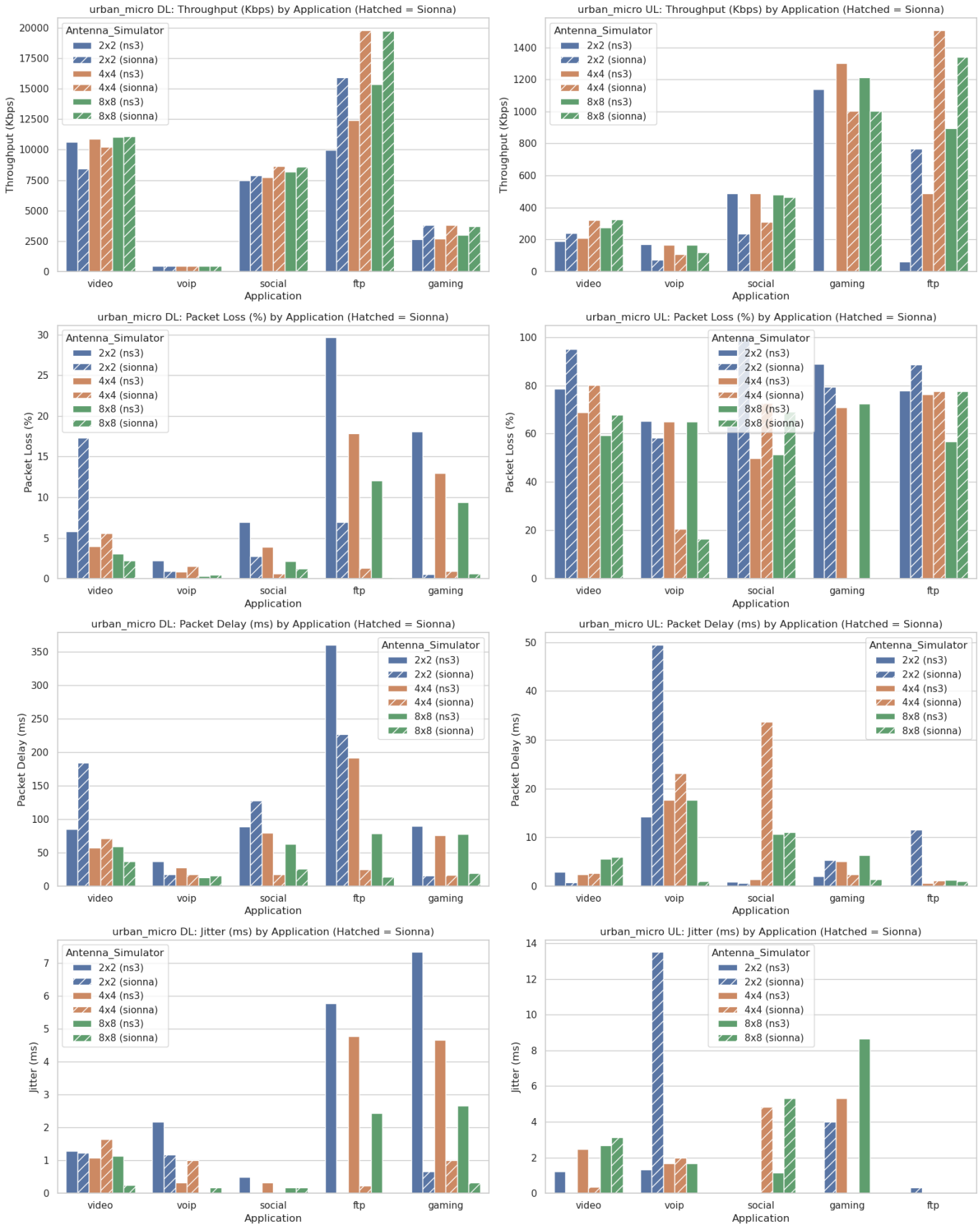


Fig. 12. Urban micro Application-related metrics by traffic type, direction, and gNB array size.

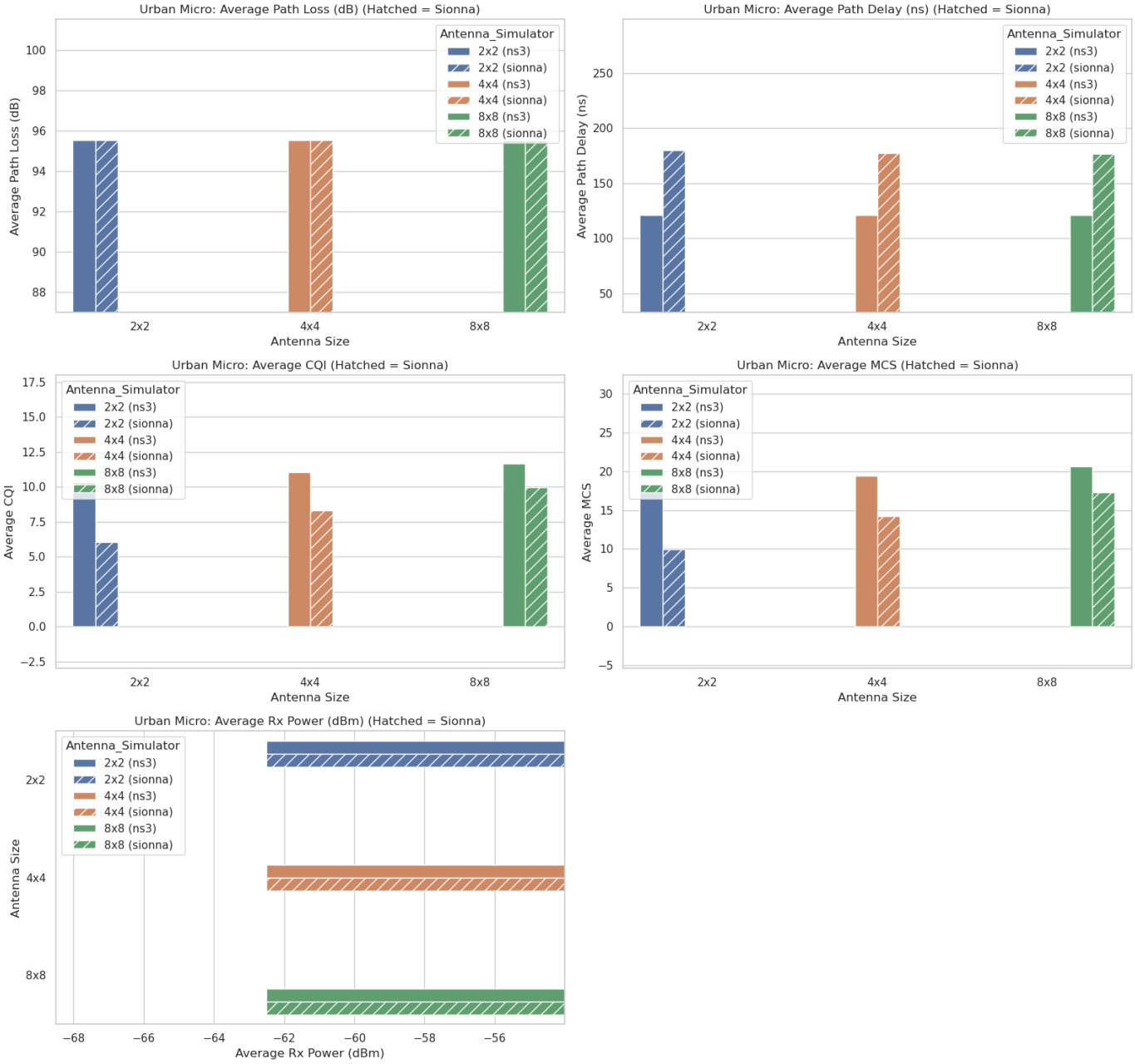


Fig. 13. Urban micro propagation-related metrics by channel mode and gNB array size.

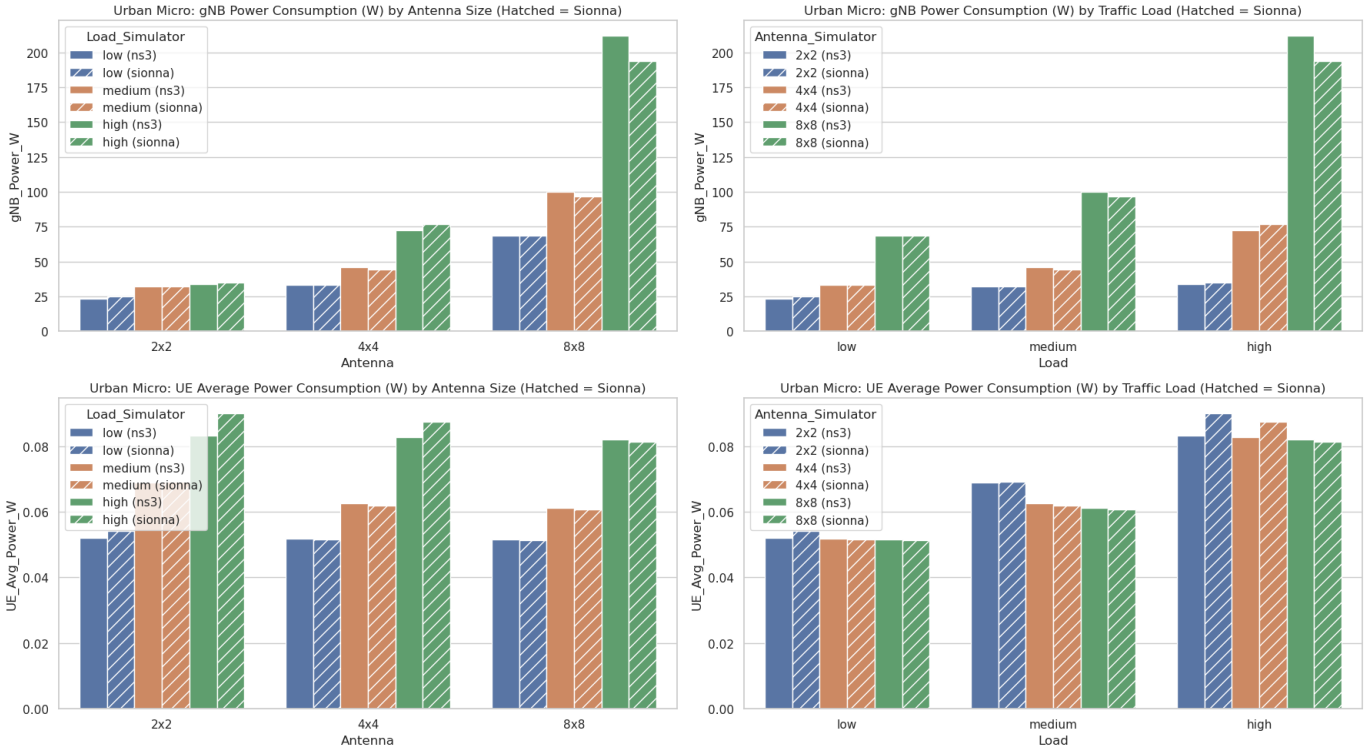


Fig. 14. Urban micro energy consumptions of gNB and UEs by array size, and traffic load.